GUÍA DE ESTUDIO · PARCIAL TEÓRICO/PRÁCTICO

Programación 3

POO en Java, contratos, excepciones, patrones de diseño y arquitectura en capas

Resumen profundo orientado a la práctica · ejercicios resueltos paso a paso con salida real · cuadros comparativos · términos clave · autoevaluación · cronograma lineal hasta el examen del **jueves 8 de octubre de 2026**

Unidad	Temas (clases del material)
1	Fundamentos de POO: objetos, clases, UML, encapsulamiento, static, final, relaciones (Clases 1 y 2)
2	Herencia y clases abstractas, delegación, SOLID (Clase 3)
3	Polimorfismo, Liskov, contratos y responsabilidades (Clase 4)
4	Interfaces, doble envío, aserciones e invariantes (Clase 5 y Proceso disciplinado)
5	Excepciones y clonación (Clase 6 y Clonación)
6	Patrones: Singleton, Factory, Template Method, Decorator, Observer, State, Frameworks
7	Arquitectura en capas (3 capas, niveles, MVC)

NOTA Todos los programas Java de esta guía fueron **compilados y ejecutados** antes de incluirlos: las salidas que ves son reales, no inventadas. Lo que no sale de tus apuntes está marcado como *extra* para que sepas qué es tuyo y qué es mío.

Índice

Cómo usar esta guía (y cronograma lineal) **4**

Unidad 1 · Fundamentos de POO (Clases 1 y 2) **5**

Ejercicio 1.1 · Paso de parámetros: ¿qué imprime? . . . 8
Ejercicio 1.2 · static y final: número de cliente único . . . 9
Ejercicio 1.3 · Encapsular y validar (get/set) . . . 9
Ejercicio 1.4 · Clasificar relaciones UML . . . 10

Unidad 2 · Herencia y clases abstractas (Clase 3) **11**

Ejercicio 2.1 · Cuentas bancarias: método abstracto y polimorfismo . . . 13
Ejercicio 2.2 · Error de compilación con super . . . 14
Ejercicio 2.3 · Reducir el acceso al sobreescribir . . . 14
Ejercicio 2.4 · ¿Herencia o delegación? . . . 15

Unidad 3 · Polimorfismo, Liskov y contratos (Clase 4) **16**

Ejercicio 3.1 · ¿Qué método y qué atributo responden? . . . 18
Ejercicio 3.2 · ¿Compila el acceso protected? . . . 19
Ejercicio 3.3 · Definir el contrato de un método . . . 19
Ejercicio 3.4 · Pre y postcondiciones al sobreescribir (Liskov) . . . 19

Unidad 4 · Interfaces, doble envío y aserciones (Clase 5 y Proceso disciplinado) **21**

Ejercicio 4.1 · Piedra, Papel o Tijera con doble envío . . . 23
Ejercicio 4.2 · Segregar una interfaz "pesada" (ISP) . . . 24
Ejercicio 4.3 · Implementar Comparable para ordenar . . . 24
Ejercicio 4.4 · Aserciones: pre, post e invariante de la clase Cuenta . . . 25
Ejercicio 4.5 · Identificar pre, post e invariante + invariante de bucle . . . 26

Unidad 5 · Excepciones y clonación (Clase 6 y Clonación) **27**

Ejercicio 5.1 · Trazar try / catch / finally con return . . . 30
Ejercicio 5.2 · Excepción propia con datos + orden de los catch . . . 31
Ejercicio 5.3 · Copia superficial vs profunda de una pila . . . 32
Ejercicio 5.4 · ¿Qué tipo de excepción usar? . . . 32

Unidad 6 · Patrones de diseño y frameworks **34**

Ejercicio 6.1 · Singleton y Factory (verificados) . . . 38
Ejercicio 6.2 · Template Method con hook: el motor . . . 39
Ejercicio 6.3 · Decorator: vehículos con motores (evitar la explosión de clases) . . . 40
Ejercicio 6.4 · Observer: termómetro y pantallas . . . 40
Ejercicio 6.5 · State: el vehículo con contacto . . . 41
Ejercicio 6.6 · Elegir el patrón correcto . . . 42

Unidad 7 · Arquitectura en capas **44**

Ejercicio 7.1 · ¿En qué capa va cada cosa? 46

Atlas de comparaciones (repaso transversal) **47**

Hoja de última hora: frases para recordar **49**

Simulacro de parcial (90 minutos) **49**

Clave de respuestas **51**

Respuestas de las autoevaluaciones **51**

Glosario de términos clave (A–Z) **56**

Cómo usar esta guía (y cronograma lineal)

Esta guía está pensada para que **estudies haciendo**, no solo leyendo. Cada unidad sigue el mismo camino, así no te perdés:

- **Resumen profundo**: lo que tenés que entender, en lenguaje simple.
- **Términos clave** y **cuadros comparativos**: para fijar las diferencias que más se confunden.
- **Ejercicios resueltos paso a paso**: primero intentá resolverlos tapando la solución, después compará.
- **Autoevaluación**: preguntas para saber si realmente lo dominás (respuestas al final).

> **TIP DE PARCIAL** En un parcial teórico/práctico casi siempre piden dos cosas: **(1)** explicar con tus palabras un concepto y compararlo con otro, y **(2)** leer o escribir un código corto y decir qué imprime o qué falla. Esta guía entrena justo eso.

Cronograma lineal: del jueves 1 al jueves 8 de octubre

Una unidad por día, siempre en el mismo orden: **leer → ejercicios → autoevaluación**. Calculá de 2 a 3 horas por día. El día 7 es para integrar todo y el día 8 (el examen) solo repasás liviano.

Día	Fecha	Unidad	Qué hacer (en este orden)	Hecho
1	Jue 1/10	U1 · Fundamentos de POO	Leer resumen → E1.1 a E1.4 → autoevaluación U1. Meta: dominar objeto vs clase, paso por valor, static/final y las 4 relaciones UML.	☐
2	Vie 2/10	U2 · Herencia y abstractas	Repasar 5 min de U1 → leer U2 → E2.1 a E2.4 → autoevaluación. Meta: super, sobreescritura, abstract, herencia vs delegación.	☐
3	Sáb 3/10	U3 · Polimorfismo y contratos	Leer U3 → E3.1 a E3.4 → autoevaluación. Meta: late binding, qué miembro responde, pre/postcondiciones, Liskov.	☐
4	Dom 4/10	U4 · Interfaces y aserciones	Leer U4 → E4.1 a E4.5 → autoevaluación. Meta: interfaz vs abstracta, doble envío, assert, invariantes.	☐
5	Lun 5/10	U5 · Excepciones y clone	Leer U5 → E5.1 a E5.4 → autoevaluación. Meta: orden de catch, finally, throws, copia superficial vs profunda.	☐
6	Mar 6/10	U6 · Patrones de diseño	Leer U6 → E6.1 a E6.6 → autoevaluación. Meta: reconocer cada patrón por su problema y escribirlo.	☐
7	Mié 7/10	U7 · Capas + repaso general	Leer U7 → E7.1 → Atlas de comparaciones → Simulacro completo (cronometrado, 90 min) → corregir.	☐
8	Jue 8/10	EXAMEN	Solo: hoja de última hora, glosario y las "trampas clásicas". Dormí bien y llegá con tiempo.	★

> **OJO** Si un día te atrasás, **no** acumules: recortá los ejercicios más largos (E6.x y E5.x son los más densos) pero no te saltes la autoevaluación. Es lo que más rinde por minuto invertido.

Método de estudio para los ejercicios de código

Paso 1. Leé el código **sin ejecutarlo** y escribí en un papel qué imprime.

Paso 2. Anotá en cada línea qué hace (quién es el receptor, qué tipo tiene la variable y qué clase tiene el objeto real).

Paso 3. Compará con la solución de la guía. Si no coincide, buscá **en qué paso** te equivocaste (casi siempre es un concepto, no un detalle).

Paso 4. Volvé a escribir la solución de memoria al día siguiente.

Unidad 1 · Fundamentos de POO (Clases 1 y 2)

Esta unidad es la base de todo lo demás. Si acá tenés claro **qué es un objeto, qué es una clase y cómo se comunican**, el resto de la materia (herencia, polimorfismo, patrones) se vuelve mucho más fácil.

1. Una nueva forma de pensar

En el paradigma **estructurado** pensamos en datos y algoritmos: descomponemos el problema en subproblemas hasta llegar a pasos simples. En el paradigma **orientado a objetos (POO)** buscamos **entidades independientes que colaboran**: cada una tiene habilidades y responsabilidades propias, y se piden cosas enviándose **mensajes**.

> **DEFINICIÓN** Un **paradigma** es un conjunto de elementos y reglas que brinda un marco para diseñar y construir programas (idea de Thomas Kuhn). La POO se apoya en cuatro pilares: **abstracción, encapsulamiento, herencia y polimorfismo**.

	Paradigma estructurado	Paradigma orientado a objetos
Los sistemas contienen...	Datos y algoritmos	Objetos que colaboran
Los programas...	Manipulan datos	Reparten responsabilidades entre objetos
Se construyen con...	Asignación, secuencia, repetición, condicional	Clases y jerarquías de herencia
Organización	Por procedimientos / funciones	Por clases
Idea fuerza	Dividir el problema en pasos	Alan Kay: todo es un objeto; los objetos se comunican enviando y recibiendo mensajes

2. Objeto, clase e instanciación

Un **objeto** representa algo del problema que estamos resolviendo y tiene tres cosas: **identidad** (es único), **estado** y **comportamiento**.

- **Estado**: los valores de sus variables de instancia (atributos) en un instante dado. Conviene que sea privado.
- **Comportamiento**: lo que sabe hacer. Se define por el conjunto de mensajes que entiende, su **protocolo**.
- **Método**: la implementación de un mensaje. Es privado del objeto: quien envía el mensaje no sabe ni le importa *cómo* se resuelve.

Un método puede hacer básicamente tres cosas: **modificar el estado** del receptor, **colaborar con otros objetos** o **retornar y terminar**. La sintaxis de un mensaje es `receptor.mensaje(parámetros)`.

Una **clase** es una descripción de un grupo de objetos con propiedades comunes, comportamiento similar y la misma forma de relacionarse. Cumple tres roles: agrupar el comportamiento común, definir la "forma" y **crear instancias**. Analogía útil: la clase es el **molde de tortas** y los objetos son las **tortas**.

¿Qué pasa cuando escribís `Cuenta c = new Cuenta();`?

Paso 1. Se reserva memoria en el **heap** para el objeto (y para la variable).

Paso 2. Se inicializan los atributos con sus **valores por defecto** (0, false, null...).

Paso 3. Se ejecuta el **constructor**.

Paso 4. Se asigna a la variable la **referencia** al nuevo objeto.

> **OJO** Las variables de tipo no primitivo **no contienen al objeto, contienen una referencia** (pensá en un puntero). Por eso dos variables pueden apuntar al mismo objeto.

Constructores

- Tienen el **mismo nombre que la clase** y **no tienen tipo de retorno** (no son métodos).
- Pueden tener cero o más parámetros y se pueden **sobrecargar**.
- Con `this(...)` un constructor puede invocar a otro de la misma clase (debe ser la primera sentencia).

Paso de parámetros: siempre por valor

En Java el pasaje de parámetros es **siempre por valor**. Con tipos primitivos se copia el valor. Con objetos se copia **la referencia**: el método puede *modificar el estado* del objeto al que apunta, pero no puede hacer que la variable del que llama apunte a otro objeto.

3. Abstracción y UML

Abstracción es suprimir detalles para concentrarte en lo significativo. Una abuela y un veterinario "abstraen" distinto a un mismo gato. Un diseño OO expresa, sin escribir código, cómo colaboran los objetos. **UML** (1994, Booch, Rumbaugh y Jacobson) es el lenguaje estándar para modelarlo. En el **diagrama de clases** cada clase se dibuja con tres compartimentos: nombre (mayúscula, sin espacios), atributos y métodos (como mínimo nombre, parámetros y retorno).

4. Encapsulamiento y modos de acceso

Encapsulamiento es ocultar la representación interna: se distingue el **"qué hacer"** (especificación, visible) del **"cómo hacerlo"** (implementación, oculta).

Modo	UML	Quién accede
público	`+`	Cualquier clase, esté o no relacionada
privado	`-`	Solo la implementación de la propia clase
protegido	`#`	La propia clase, sus subclases (y el mismo paquete en Java)

¿Por qué no hacer públicos los atributos? Porque si `edad` es público, cualquiera puede escribir `unaPersona.edad = -7`. La responsabilidad de **validar** debe estar en la propia clase y no repartida por todo el código. Por eso usamos métodos de acceso `getXxx()` y `setXxx()` (y el `set` valida).

> **TRAMPA CLÁSICA** En Java el control de acceso es **por clase, no por objeto**: un objeto puede leer los atributos privados de *otro objeto de la misma clase*.

5. Sobrecarga, static y final

Signatura (firma) = nombre + parámetros (cantidad, tipos y orden). Es toda la cabecera **excepto el retorno**. Con la **sobrecarga** varios métodos (o constructores) comparten nombre pero difieren en la signatura; **no se puede sobrecargar cambiando solo el tipo de retorno**.

	Miembro de instancia	Miembro `static`
Pertenece a…	Cada objeto (cada uno tiene su copia)	La clase (una sola copia compartida)
Se accede con…	`objeto.miembro`	`Clase.miembro` (no hace falta ningún objeto)
`this`	Sí existe	No existe (no hay objeto)
Un método puede usar…	Instancia y static	Solo miembros static
Uso típico	Estado propio (saldo, nombre)	Contadores, constantes, métodos utilitarios

`final` crea constantes: una vez inicializada no cambia. Se puede inicializar en la declaración o en el constructor. Si el atributo `final` es un **objeto**, no podés cambiar *a qué objeto apunta*, pero sí **modificar su estado interno**.

6. Recolector de basura y `finalize`

Cuando ninguna referencia apunta a un objeto, el **Garbage Collector** lo libera: es **automático, seguro y transparente**. Desventajas: no sirve para sistemas de tiempo real y los primeros tenían bastante overhead. El método `finalize()` se invoca justo antes de liberar la memoria (similar a un destructor de C++); solo tiene sentido para liberar **recursos que no son de Java** (archivos, handles).

7. Relaciones entre clases

Esta es una de las preguntas más frecuentes. Tres preguntas te ayudan a clasificar: **¿Una clase tiene a la otra como atributo? ¿Pueden existir por separado? ¿Se comparten?**

Relación	Símbolo UML	Idea	Vida	Ejemplo del apunte
Asociación	Línea	Se "conocen"; suele ser bidireccional	Independientes	Persona ↔ Sucursal
Agregación	Rombo **blanco** en el "todo"	Parte-todo débil; la parte se puede **compartir**	La parte sobrevive si el todo se destruye	Empresa–Cliente, Persona–Categoría
Composición	Rombo **negro** en el "todo"	Parte-todo fuerte; la parte **no se comparte**	La parte muere con el todo	Empresa–Empleado, Persona–Domicilio
Dependencia	Flecha punteada	Solo **usa** al otro en una operación; no lo tiene como atributo	Independientes	Persona usa Postulación (un método la crea y la devuelve)

8. Bloques de inicialización

Un **bloque de inicialización** `{ ... }` dentro de la clase (fuera de métodos y constructores) inicializa campos de instancia en inicializaciones no triviales. Un **bloque estático** `static { ... }` hace lo mismo para campos estáticos (por ejemplo, armar un arreglo estático) y solo puede referirse a miembros estáticos.

Términos clave de la Unidad 1

Término	En una frase
Objeto	Entidad con identidad, estado y comportamiento
Clase	Molde que define estructura y comportamiento de sus instancias
Instanciación	Crear un objeto con `new` a partir de una clase
Mensaje / método	Pedido enviado a un objeto / código que lo resuelve
Protocolo	Conjunto de mensajes que un objeto entiende
Referencia / heap	Dirección del objeto / zona de memoria donde viven los objetos
Constructor	Inicializa el objeto; mismo nombre que la clase, sin retorno
Encapsulamiento	Ocultar la implementación y mostrar solo la interfaz
Signatura	Nombre + parámetros (sin el tipo de retorno)
`static` / `final`	Compartido por la clase / no modificable
Agregación vs composición	Rombo blanco, parte compartible / rombo negro, vida ligada

Ejercicio 1.1 · Paso de parámetros: ¿qué imprime?

```
class Persona {
    String nombre;
    Persona(String nombre) { this.nombre = nombre; }
}
public class Main {
    static void cambiar(Persona p) {
        p.nombre = "Ana";          // (1)
        p = new Persona("Zoe");    // (2)
        p.nombre = "Lucia";        // (3)
    }
    static void sumar(int k) { k = k + 10; }
    public static void main(String[] args) {
        Persona x = new Persona("Juan");
        Persona y = x;
        cambiar(x);
        System.out.println(x.nombre);
        System.out.println(y == x);
        int n = 5;
        sumar(n);
        System.out.println(n);
    }
}
```

Resolución paso a paso

Paso 1. `x` guarda una referencia al objeto "Juan". `y = x` copia **la referencia**: ahora `x` e `y` apuntan al **mismo** objeto.

Paso 2. Al llamar `cambiar(x)`, el parámetro `p` recibe una **copia de la referencia**. Entonces `p` y `x` apuntan al mismo objeto.

Paso 3. Línea (1): `p.nombre = "Ana"` modifica el objeto compartido. Ahora ese objeto se llama "Ana" (lo ven `x`, `y` y `p`).

Paso 4. Línea (2): `p = new Persona("Zoe")` hace que la **copia local** `p` apunte a un objeto nuevo. `x` no se entera, sigue apuntando al original.

Paso 5. Línea (3): modifica solo el objeto nuevo ("Zoe" pasa a "Lucia"), que se pierde al terminar el método.

Paso 6. Por eso `x.nombre` es **"Ana"** y `y == x` es **true** (nunca dejaron de ser el mismo objeto).

Paso 7. Con el `int`: `sumar` recibe una copia del valor 5; cambiar `k` no toca a `n`. Imprime **5**.

Salida real (verificada ejecutando el código):

```
Ana
true
5
```

> **TIP DE PARCIAL** Regla de oro: *Java copia siempre lo que hay en la variable.* Si es un primitivo, copia el valor; si es un objeto, copia la **dirección**. Por eso podés modificar el objeto pero no reasignar la variable del que llama.

Ejercicio 1.2 · static y final: número de cliente único

```
class Cliente {
    private static int siguienteNro = 1;   // compartido por todos
    private final int nro;                 // constante por objeto
    private String apellido;
    public Cliente(String apellido) {
        this.nro = siguienteNro++;
        this.apellido = apellido;
    }
    public int getNro() { return nro; }
    public static int getSiguienteNro() { return siguienteNro; }
}
// en main:
Cliente c1 = new Cliente("Gomez");
Cliente c2 = new Cliente("Pereira");
System.out.println(c1.getNro());
System.out.println(c2.getNro());
System.out.println(Cliente.getSiguienteNro());
final Persona p = new Persona("Juan");
p.nombre = "Pedro";
System.out.println(p.nombre);
```

Paso 1. Hay **una sola** copia de `siguienteNro` (es `static`), que vale 1 al cargar la clase.

Paso 2. Al crear `c1`: `nro = siguienteNro++` asigna 1 y luego incrementa el contador a 2. Al crear `c2`: asigna 2 y el contador queda en 3.

Paso 3. `nro` es `final`: se asigna **una vez en el constructor** y ya no cambia. Cada objeto tiene el suyo (1 y 2).

Paso 4. `Cliente.getSiguienteNro()` se invoca sobre la **clase**, sin objeto, y devuelve 3.

Paso 5. Con `final Persona p`: la referencia `p` no puede reasignarse, pero el **objeto** sí puede cambiar su estado. Por eso `p.nombre = "Pedro"` es válido y se imprime Pedro.

Salida real (verificada ejecutando el código):

```
1
2
3
Pedro
```

> **TRAMPA CLÁSICA** Si dentro de un método `static` escribís `this.nro`, **no compila**: en un método estático no hay objeto, por lo tanto no existe `this`.

Ejercicio 1.3 · Encapsular y validar (get/set)

Pasá `Persona` con atributos públicos a una clase que **no permita edades negativas**. Solución:

```
class Persona {
    private String nombre;
    private int edad;
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        setEdad(edad);                     // el constructor reutiliza la validación
    }
    public int getEdad() { return edad; }
    public void setEdad(int edad) {
        if (edad < 0 || edad > 130)
            throw new IllegalArgumentException("Edad invalida: " + edad);
        this.edad = edad;
    }
}
// main: p = new Persona("Ana", 30); p.setEdad(31); p.setEdad(-7);
```

Paso 1. Se vuelven `private` los atributos: nadie de afuera puede hacer `p.edad = -7`.

Paso 2. Se agregan `getEdad()` (consulta) y `setEdad()` (modificación **con control**).

Paso 3. La validación queda en **un solo lugar** (la clase dueña del dato). El constructor llama a `setEdad` para no duplicar la regla.

Paso 4. Si el valor es inválido, se lanza una excepción y el estado del objeto **no cambia**.

Salida real (verificada ejecutando el código):

```
31
Edad invalida: -7
31
```

Ejercicio 1.4 · Clasificar relaciones UML

Clasificá cada caso como asociación, agregación, composición o dependencia y justificá.

Caso	Respuesta	Justificación
Un **Auto** tiene un **Motor** que se fabrica con él y no existe fuera del auto	**Composición**	La vida del motor está ligada a la del auto y no se comparte
Un **Curso** tiene **Alumnos**; si se cierra el curso, los alumnos siguen existiendo	**Agregación**	Las partes sobreviven al todo y podrían estar en otros cursos
Un método de **Cajero** recibe una **Tarjeta** por parámetro, la lee y termina	**Dependencia**	Solo la usa en una operación; no es atributo
Cliente conoce a su **Vendedor** y el **Vendedor** conoce a sus Clientes	**Asociación**	Bidireccional; ninguna depende de la vida de la otra
Persona y su **Domicilio** (si no hay persona, el domicilio debe desaparecer)	**Composición**	Ejemplo exacto del apunte: parte inseparable

Truco de decisión: (1) ¿es atributo? No → *dependencia*. (2) ¿se destruyen juntos? Sí → *composición*. (3) ¿la parte es de un todo y se puede compartir? Sí → *agregación*. (4) Si no, es una *asociación* general.

Autoevaluación: ¿qué tan firme estás?

Contestá primero **sin mirar** las respuestas (están al final de la guía). Si dudás en una, volvé al resumen.

- ☐ **1.** Diferenciá **clase** de **objeto** con un ejemplo propio.
- ☐ **2.** ¿Cuáles son los 4 pasos que ocurren al ejecutar `new`?
- ☐ **3.** Verdadero o falso: en Java los objetos se pasan por referencia a los métodos. Justificá.
- ☐ **4.** ¿Por qué no conviene hacer públicos los atributos? ¿Quién debe validar los datos?
- ☐ **5.** ¿Se puede sobrecargar un método cambiando solo el tipo de retorno? ¿Por qué?
- ☐ **6.** ¿Qué puede hacer y qué no un método `static`?
- ☐ **7.** Si un atributo es `final` y de tipo objeto, ¿podés cambiar el estado del objeto? ¿y reasignarlo?
- ☐ **8.** Diferencia entre agregación y composición, con símbolos UML.
- ☐ **9.** ¿Para qué sirve `finalize()` y por qué casi nunca se redefine?
- ☐ **10.** Un objeto A puede leer el atributo privado de otro objeto B, ¿cuándo?

Unidad 2 · Herencia y clases abstractas (Clase 3)

La herencia permite crear clases nuevas a partir de otras ya existentes. Es, después de la composición, el mecanismo más usado para lograr **reutilización** y **extensibilidad**. Pero usarla bien implica saber *cuándo sí y cuándo no*.

1. Concepto de herencia

La herencia implementa la relación **"es un"**: un Gato **es un** Animal. La subclase obtiene dos cosas: la **herencia de contrato o tipo** (se la puede usar donde se espera la superclase) y la **herencia de implementación** (campos y métodos accesibles). En Java se indica con `extends`, y toda clase que no declare padre hereda de `java.lang.Object`.

> **DEFINICIÓN** Una subclase puede **agregar** características y comportamiento, y también **reemplazar o modificar** el comportamiento heredado. Los miembros heredados no se vuelven a escribir, salvo para sobreescribirlos u ocultarlos.

2. Sobreescritura, sobrecarga y ocultamiento

	Sobrecarga (overload)	Sobreescritura (override)	Ocultamiento (hiding)
Qué es	Mismo nombre, distinta signatura	Misma signatura que la del padre, nueva implementación	Atributo del hijo con mismo nombre que el del padre
Dónde	En la misma clase o en la subclase	Solo en una subclase	Solo en una subclase
Se resuelve...	Al compilar (por parámetros)	Al ejecutar (según el objeto real)	Al compilar (según el **tipo de la variable**)
Objetivo	Ofrecer variantes del mismo servicio	Cambiar la implementación heredada	Casi nunca conviene

Con `super` accedés a un miembro del padre que quedó **oculto** (atributo) o **sobreescrito** (método o constructor): `super.metodo()` o `super(...)`.

3. Constructores en clases extendidas

Solo la superclase sabe inicializar correctamente su propio estado. Por eso el constructor de la subclase debe **delegar** invocando a un constructor del padre con `super(...)`, y debe ser **la primera sentencia**. Si no escribís nada, Java inserta un `super()` sin argumentos: si el padre **no tiene** constructor sin parámetros, el código **no compila** (ver Ejercicio 2.2).

4. Especificadores de acceso al redefinir

Un método que redefine a otro puede cambiar su acceso, pero **solo para hacerlo más accesible**: `private → protected → public`, nunca al revés. Si lo hicieras menos accesible romperías el contrato del padre, porque una instancia de la subclase ya no podría usarse en lugar del padre.

5. `final` en clases, métodos y variables

Se aplica a...	Significado
Clase `final`	No puede extenderse (punto final de la jerarquía)
Método `final`	No puede redefinirse en una subclase
Variable `final`	Su valor no puede cambiar una vez inicializada (constante)

6. Herencia vs delegación

La **delegación** ocurre cuando una clase **contiene** (como atributos) instancias de otra a la que delega parte de su trabajo. Es la opción indicada cuando necesitás asociar clases pero el principio de Liskov no se cumple: un Vehículo **tiene un** Motor, pero un Alumno **es una** Persona.

	Herencia	Delegación / composición
Relación	"es un"	"tiene un" / "usa un"
Reutilización	"Todo o nada": heredás todo	**Selectiva**: elegís qué delegar
Transparencia	Caja **blanca** (la subclase ve la implementación)	Caja **negra** (solo se usa la interfaz)
Escritura	Declarativa y corta	Más código (hay que escribir las llamadas)
Cuándo conviene	Cuando el "es un" es claro y obvio	Cuando no hace falta tratar al objeto como el padre (upcasting)

> **TIP DE PARCIAL** Frase para recordar: **"Preferí composición a herencia, salvo que el *es un* sea obvio"**. Pero atención: solo es buena elección si simplifica más de lo que complica.

7. Clases y métodos abstractos

Algunos conceptos son demasiado generales como para tener instancias: no existe "una comida" en abstracto (¿qué forma, color, sabor tendría?). Eso es una **clase abstracta** (`abstract`): **no se puede instanciar** y existe para ser **extendida** por clases concretas, definiendo una interfaz de comportamiento común.

- Puede tener **métodos abstractos** (sin cuerpo): obligan a las subclases concretas a implementarlos.
- Puede tener **métodos concretos** y atributos; un método concreto puede invocar a uno abstracto (idea base del patrón Template Method).
- Si una subclase no implementa **todos** los métodos abstractos, también debe declararse abstracta.

Ejemplo del apunte (`Ordenable`): la clase abstracta implementa el algoritmo de ordenación por selección y deja abstracto `int compare(Ordenable b)`, porque el criterio de comparación recién se conoce en la clase concreta. El método concreto `p_menor(...)` invoca al abstracto `compare(...)`.

8. Diseñar para extender: generalización y especialización

	Generalización (de abajo hacia arriba)	Especialización (de arriba hacia abajo)
Punto de partida	Clases diseñadas por separado	Clases ya existentes en el sistema
Pasos	Descubrir lo común → abstraerlo en una superclase → quitar lo común de las subclases → verificar el "es un"	Buscar una clase con características comunes → si cumple el "es un", crear la nueva clase a partir de ella

9. SOLID (visión general)

Acrónimo de Robert C. Martin: cinco principios de diseño OO para que el código sea fácil de mantener y extender, muy ligados a **alta cohesión y bajo acoplamiento**. En tus apuntes aparecen tres de ellos:

Letra	Principio	Dónde aparece en el material
S	Responsabilidad única	*extra*: cada clase debería tener un solo motivo para cambiar (se relaciona con capas)
O	Abierto/cerrado	*extra*: abierto a extensión, cerrado a modificación (Decorator, Template)
L	**Sustitución de Liskov**	Polimorfismo (U3), Decorator
I	**Segregación de interfaces**	Interfaces (U4)

Letra	Principio	Dónde aparece en el material
D	**Inversión de dependencias**	Template Method (la clase base llama a las hijas)

10. La clase `Object` y `toString`

`Object` es la raíz de la jerarquía y aporta métodos como `toString()`, `equals()` y `clone()`. Redefinir `toString()` te permite devolver una descripción legible del objeto.

Términos clave de la Unidad 2

Término	En una frase
Herencia	Mecanismo "es un": la subclase hereda contrato e implementación
Sobreescritura	Misma firma en la subclase para cambiar la implementación
Ocultamiento	Atributo del hijo que "tapa" al del padre (se resuelve por tipo de variable)
`super`	Referencia a lo heredado: `super.m()` y `super(...)`
Clase abstracta	No instanciable; define comportamiento común y métodos abstractos
Método abstracto	Sin cuerpo; obliga a las subclases concretas a implementarlo
Delegación	Una clase contiene a otra y le reenvía parte del trabajo
Generalización / especialización	Crear la jerarquía desde abajo / desde arriba
Principio "es un"	Condición para que una herencia sea válida

Ejercicio 2.1 · Cuentas bancarias: método abstracto y polimorfismo

El banco tiene tres tipos de cuenta que comparten saldo, depósito y extracción, pero cada una **restringe distinto las extracciones**: Caja de ahorro (máximo de extracciones mensuales, sin descubierto), Corriente (permite descubierto) y Universitaria (máximo $100 por extracción). Diseñá la jerarquía.

Paso 1. Lo común se sube a una clase abstracta `Cuenta`: el saldo, `depositar` y `extraer`.

Paso 2. Lo que varía (la regla para permitir una extracción) se encapsula en un método **abstracto** `puedeExtraer(monto)`.

Paso 3. `extraer` es un método concreto que usa esa regla: si `puedeExtraer` da true, descuenta el monto.

Paso 4. Cada subclase concreta implementa `puedeExtraer` con su propia regla. La Caja de ahorro además sobreescribe `extraer` para contar extracciones usando `super.extraer(...)`.

```
abstract class Cuenta {
    protected double saldo;
    public void depositar(double monto) { saldo += monto; }
    public boolean extraer(double monto) {
        if (puedeExtraer(monto)) { saldo -= monto; return true; }
        return false;
    }
    protected abstract boolean puedeExtraer(double monto);
    public double getSaldo() { return saldo; }
}
class CajaAhorro extends Cuenta {
    private static final int MAX_EXTRACCIONES = 2;
    private int extracciones;
    protected boolean puedeExtraer(double monto) {
        return extracciones < MAX_EXTRACCIONES && monto <= saldo;
    }
    @Override public boolean extraer(double monto) {
        boolean ok = super.extraer(monto);
        if (ok) extracciones++;
```

```
        return ok;
    }
}
class CuentaCorriente extends Cuenta {
    private double descubierto;
    public CuentaCorriente(double d) { descubierto = d; }
    protected boolean puedeExtraer(double monto) { return monto <= saldo + descubierto; }
}
class CuentaUniversitaria extends Cuenta {
    protected boolean puedeExtraer(double monto) { return monto <= 100 && monto <= saldo; }
}
// main: para cada cuenta -> depositar(100); extraer(120); extraer(60);
Cuenta[] cuentas = { new CajaAhorro(), new CuentaCorriente(50), new CuentaUniversitaria() };
```

Salida real (verificada ejecutando el código):

```
CajaAhorro: false, true, saldo=40.0
CuentaCorriente: true, false, saldo=-20.0
CuentaUniversitaria: false, true, saldo=40.0
```

Paso 1. Caja de ahorro: tiene 100. `extraer(120)` → 120 > saldo, **false**. `extraer(60)` → permitido, saldo 40, **true**.

Paso 2. Corriente (descubierto 50): `extraer(120)` → 120 ≤ 100+50, **true**, saldo −20. `extraer(60)` → 60 > −20+50 = 30, **false**.

Paso 3. Universitaria: `extraer(120)` → supera los $100, **false**. `extraer(60)` → ok, saldo 40.

Paso 4. Fijate que el **mismo código** (`c.extraer(...)` en un `for` sobre `Cuenta[]`) produce resultados distintos según el objeto real. Eso es **polimorfismo**, y lo profundizamos en la Unidad 3.

> **TIP DE PARCIAL** Si en el parcial te piden "modelá X con una clase abstracta", seguí siempre esta receta: **(1)** identificá lo común, **(2)** identificá lo que varía, **(3)** lo común va concreto, lo que varía va **abstracto**.

Ejercicio 2.2 · Error de compilación con `super`

¿Compila este código? Si no, explicá por qué y cómo se arregla.

```
class Vehiculo {
    private String patente;
    public Vehiculo(String patente) { this.patente = patente; }
}
class Auto extends Vehiculo {
    private int puertas;
    public Auto(int puertas) { this.puertas = puertas; }
}
```

Mensaje real del compilador:

```
Main.java:7: error: constructor Vehiculo in class Vehiculo cannot be applied to given types;
    public Auto(int puertas) { this.puertas = puertas; }
                             ^
  required: String
  found:    no arguments
  reason: actual and formal argument lists differ in length
```

Paso 1. Si el constructor de `Auto` no llama a `super(...)`, Java agrega implícitamente `super()` (sin argumentos).

Paso 2. `Vehiculo` solo tiene un constructor que **exige** un `String`. No existe `Vehiculo()`, entonces el compilador falla.

Paso 3. Arreglo: `public Auto(String patente, int puertas) { super(patente); this.puertas = puertas; }`. Así se delega en el padre la construcción del estado heredado.

Ejercicio 2.3 · Reducir el acceso al sobreescribir

```
class Padre { public void hablar() { System.out.println("Padre"); } }
class Hijo extends Padre { private void hablar() { System.out.println("Hijo"); } }
```

Mensaje real del compilador:

```
Main.java:2: error: hablar() in Hijo cannot override hablar() in Padre
  attempting to assign weaker access privileges; was public
```

Paso 1. El método del padre es `public`. El hijo lo redefine como `private`: **menos accesible**.

Paso 2. Eso rompería el principio de sustitución: `Padre p = new Hijo(); p.hablar();` debería funcionar siempre.

Paso 3. Regla: al redefinir se puede mantener o **ampliar** el acceso (private → protected → public), jamás reducirlo.

Ejercicio 2.4 · ¿Herencia o delegación?

Situación	Decisión	Por qué
Alumno y Persona	Herencia	Un alumno **es una** persona (se cumple Liskov)
Vehículo y Motor	Delegación	Un vehículo **tiene un** motor; no es un motor
Gato y Animal	Herencia	"Es un" claro y obvio
Pila implementada usando una Lista	Delegación	Una pila no *es una* lista (no debería exponer todos sus métodos); solo la **usa**
Telefono, TelefonoCelular, TelefonoPublico	Herencia	Comparten número/marca/sonar/llamar y cada uno agrega lo suyo

Autoevaluación: ¿qué tan firme estás?

Contestá primero **sin mirar** las respuestas (están al final de la guía). Si dudás en una, volvé al resumen.

☐ **1.** ¿Qué dos cosas hereda una subclase de su superclase?

☐ **2.** Diferenciá sobrecarga, sobreescritura y ocultamiento.

☐ **3.** ¿Para qué sirve `super(...)` en un constructor y dónde debe ir?

☐ **4.** Un método público del padre, ¿puede redefinirse como protegido? ¿Y uno protegido como público?

☐ **5.** Explicá los tres usos de `final`.

☐ **6.** ¿Cuándo conviene delegación en lugar de herencia?

☐ **7.** ¿Se puede instanciar una clase abstracta? ¿Puede tener métodos concretos?

☐ **8.** Si una subclase no implementa un método abstracto del padre, ¿qué pasa?

☐ **9.** Describí los pasos de la generalización.

☐ **10.** ¿Qué clase es la raíz de la jerarquía en Java y qué método se suele redefinir para describir un objeto?

Unidad 3 · Polimorfismo, Liskov y contratos (Clase 4)

Esta es la unidad "bisagra": explica por qué la herencia es tan potente (polimorfismo) y cómo evitar que se vuelva peligrosa (contratos). Casi seguro te toman un ejercicio de **"¿qué método/atributo responde?"**.

1. Principio de sustitución de Liskov

> **LISKOV** "Debe ser posible utilizar cualquier objeto instancia de una subclase en lugar de cualquier objeto instancia de su superclase **sin que la semántica del programa se vea afectada**." En criollo: **donde va un padre, puede ir un hijo.**

2. Tipos de polimorfismo

Polimorfismo significa "que posee varias formas". En el apunte se distinguen tres:

Tipo	Qué es	Ejemplo
De **sobrecarga**	Un operador cambia su comportamiento según los operandos	`+` suma enteros y concatena cadenas
Paramétrico (según el apunte)	Varios métodos con el mismo nombre pero distintos parámetros; se elige automáticamente según el tipo de dato	`int addition(int,int)`, `float addition(float,float)`
De **inclusión** (redefinición / subtipado)	Redefinir un método en una subclase; se puede llamar a un método sin conocer el tipo exacto del objeto, usando la clase base como interfaz común	`animal.emitirSonido()` donde `animal` puede ser Perro o Gato

El que más importa en la materia es el de **inclusión**: permite tratar objetos de tipos derivados como si fueran del tipo base, y cada uno responde a su manera.

3. Enlace tardío (late binding)

Ligadura es conectar una llamada a un método con su código. Puede ser **temprana** (en compilación) o **tardía** (en ejecución). Con polimorfismo, en compilación no se sabe cuál será el objeto real que ejecutará el método, así que se resuelve **en tiempo de ejecución**.

4. ¿Qué responde: la variable o el objeto? (acceso a miembros)

Suponé `Padre p = new Hijo();`. Hay dos tipos en juego: el **tipo declarado** de la variable (Padre) y la **clase real** del objeto (Hijo). La regla de oro:

Qué accedés	Quién decide	Consecuencia
Métodos `p.metodo()`	La **clase real del objeto** (Hijo)	Se ejecuta la versión sobreescrita (late binding)
Atributos `p.atributo`	El **tipo declarado** de la variable (Padre)	Se lee el atributo del padre aunque el hijo lo haya ocultado
Miembros propios del hijo	Nadie: no son visibles desde `Padre`	`p.atributo3` y `p.metodo4()` **no compilan** sin un cast

> **TIP DE PARCIAL** Frase para el parcial: **"Los métodos son polimórficos, los atributos no."**

5. El verdadero significado de `protected`

Un miembro `protected` es accesible por las **subclases**, por clases del **mismo paquete** y, además, desde otra clase **a través de referencias que sean al menos del mismo tipo que la clase que accede** (su propio tipo o un subtipo). Es la parte sutil: una `FilaConPrioridades` (en otro paquete) puede acceder a `head`/`tail` de **otra `FilaConPrioridades`**, pero no a los de una `Fila` cualquiera, porque no toda `Fila` es una `FilaConPrioridades`.

6. Contratos: precondiciones, postcondiciones

Respetar la firma de un método **no alcanza** para aplicar Liskov sin problemas. Hay que documentar un **contrato**: bajo qué condiciones el método tendrá éxito y qué resultado deja. Quien llama necesita saber qué supuso quien programó el método **sin leer su implementación**.

	Precondición	Postcondición
Qué es	Condiciones que **deben ser ciertas ANTES** para que el método funcione	Condiciones que **serán ciertas DESPUÉS** (si no hubo excepción)
Quién la garantiza	El que **llama** al método	El método **llamado**
Puede referirse a…	Estado del objeto, estado de objetos asociados, parámetros de entrada	Valor de retorno y estado del objeto tras la ejecución
¿Se verifica dentro?	**No**: se supone cierta; si falla, es culpa de quien llamó	Es lo que el método se compromete a cumplir

Preguntas típicas del apunte

- **¿Un método debe verificar su precondición?** No. Lo que está en la precondición se supone cierto.
- **¿Cuándo va una excepción en el contrato?** Solo cuando, **aun cumpliéndose toda la precondición**, es imposible cumplir la postcondición. Ninguna excepción debe asociarse al incumplimiento de una precondición.
- **¿Qué incluir en la precondición?** Solo condiciones **fáciles de garantizar** por quien llama. Si verificarlas previamente es muy caro, sacala de la precondición y lanzá una excepción desde el método.
- **¿Por qué no verificar todo adentro?** Por ineficiencia: conviene **repartir responsabilidades** entre quien llama y quien implementa, dejándolo claro en el contrato.

> **CONTRATOS Y HERENCIA** Al **sobreescribir** un método y mantener Liskov, **se pueden relajar las precondiciones y fortalecer las postcondiciones**, pero nunca al revés (exigir más antes o prometer menos después).

7. Asignar responsabilidades

El apunte presenta dos técnicas con el caso del **Club social** (Club con socios; cada Socio con autorizados y facturas):

- **Técnica del experto**: la responsabilidad recae en la clase **dueña de la información**. Ej.: afiliar un socio es de `Club` (dueño de la lista de socios), registrar un autorizado es de `Socio`. No se devuelve el arreglo para que otro lo modifique: se ofrece un método.
- **Descomposición de requerimientos**: se parte un requerimiento en pasos y cada paso se asigna a la clase correspondiente. Ej. registrar una persona autorizada: (1) `Club.buscarSocio()`, (2) `Socio.existeAutorizado()`, (3) `Socio.agregarAutorizado()`.

También se organiza el sistema en **paquetes** (`interfaz`, `mundo` con Club/Socio/Factura, `test`) y los contratos se documentan con **Javadoc**.

Términos clave de la Unidad 3

Término	En una frase
Liskov	Donde va un padre puede ir un hijo sin alterar la semántica
Polimorfismo de inclusión	Un mismo mensaje, distinto comportamiento según la clase real
Ligadura tardía	La elección del método se hace al ejecutar, según el objeto
Tipo declarado vs clase real	Tipo de la variable vs clase con la que se hizo `new`
`protected`	Subclases + paquete + referencias del mismo tipo (o subtipo)
Contrato	Conjunto de suposiciones (pre) y compromisos (post) de un método
Experto	Asignar la responsabilidad a quien tiene la información

Ejercicio 3.1 · ¿Qué método y qué atributo responden?

```
class Padre {
    String atributo1 = "P1";  String atributo2 = "P2";
    String metodo1() { return "Padre.metodo1"; }
    String metodo2() { return "Padre.metodo2"; }
    String metodo3() { return "Padre.metodo3"; }
}
class Hijo extends Padre {
    String atributo1 = "H1";  String atributo3 = "H3";
    @Override String metodo1() { return "Hijo.metodo1"; }
    @Override String metodo2() { return "Hijo.metodo2"; }
    String metodo4() { return "Hijo.metodo4"; }
}
// main:
Padre p = new Hijo();
p.metodo1();   p.metodo2();   p.metodo3();
p.atributo1;   p.atributo2;
((Hijo) p).atributo1;   ((Hijo) p).metodo4();
```

Salida real (verificada ejecutando el código):

```
Hijo.metodo1
Hijo.metodo2
Padre.metodo3
P1
P2
H1
Hijo.metodo4
```

Paso 1. Tipo declarado de `p`: **Padre**. Clase real del objeto: **Hijo**.

Paso 2. `p.metodo1()` y `p.metodo2()` están **sobreescritos** en Hijo → manda el objeto real → versiones del Hijo.

Paso 3. `p.metodo3()` no fue sobreescrito → se hereda → `Padre.metodo3`.

Paso 4. `p.atributo1`: los atributos se resuelven por el **tipo de la variable** (Padre) → **"P1"**, aunque Hijo tenga el suyo.

Paso 5. `p.atributo2` solo existe en Padre → "P2". En cambio `p.atributo3` y `p.metodo4()` **no compilan**: `Padre` no los conoce.

Paso 6. Con el cast `(Hijo) p` pasamos a mirar al objeto como Hijo: ahora `atributo1` es "H1" y `metodo4()` es accesible.

> **TRAMPA CLÁSICA** El error clásico es contestar "H1" para `p.atributo1`. Recordá: **atributos por tipo de variable, métodos por objeto real**.

Ejercicio 3.2 · ¿Compila el acceso `protected`?

`Fila` (paquete A) declara `protected Nodo head`. `FilaConPrioridades` (paquete B) extiende a `Fila`. ¿Qué pasa con estos dos métodos dentro de `FilaConPrioridades`?

```
public void merge(FilaConPrioridades q) { Nodo first = q.head; /* ... */ }  // (a)
public void merge(Fila q)               { Nodo first = q.head; /* ... */ }  // (b)
```

Paso 1. (a) El parámetro `q` es del **mismo tipo que la clase que contiene `merge`** (o de un subtipo): el acceso a `head` está permitido. **Compila.**

Paso 2. (b) `q` es de tipo `Fila`, que **no es la misma clase ni una subclase** de `FilaConPrioridades`, y está en **otro paquete**. El acceso **no está permitido**: **no compila**.

Paso 3. Razón de fondo: toda `FilaConPrioridades` es una `Fila`, pero **no toda `Fila` es una `FilaConPrioridades`**.

Nota: este ejercicio sigue la explicación del apunte (no lo ejecuté porque requiere dos paquetes).

Ejercicio 3.3 · Definir el contrato de un método

Escribí pre y postcondiciones de `afiliarSocio(String cedula, String nombre)` (la clase `Club` guarda una lista de socios) y la precondición de `registrarConsumo(String nombreA, String concepto, double valor)` de `Socio`.

Paso 1. **Pensá en el "antes"**: ¿qué debe ser cierto para que el método pueda cumplir? Lista de socios ya creada; `cedula` no nula ni vacía; `nombre` no nulo ni vacío. (Que no exista ya un socio con esa cédula **no** va como precondición: es costoso de garantizar y se resuelve con una excepción).

Paso 2. **Pensá en el "después"**: o bien (a) el socio quedó afiliado al club, o bien (b) se lanzó una excepción informando que ya existía y el socio **no** quedó afiliado.

Paso 3. Para `registrarConsumo`, las suposiciones se agrupan: sobre cada **parámetro** (por ejemplo: `nombreA` no nulo ni vacío, `concepto` no nulo ni vacío, `valor > 0`), sobre el **estado del objeto** y sobre **objetos asociados**.

Paso 4. Documentá todo con Javadoc (`@param`, `@pre`, `@post`) para que el contrato sea público.

> **TIP DE PARCIAL** En un parcial, redactá cada contrato así: **"Pre: …"** y **"Post: …"**. Si el método puede fallar aun cumpliéndose la pre, agregá la excepción en la post.

Ejercicio 3.4 · Pre y postcondiciones al sobreescribir (Liskov)

El método `Padre.m(int x)` tiene como precondición `x > 0` y como postcondición "retorna un valor positivo". Para cada versión del hijo, ¿respeta Liskov?

Hijo.m(int x)	Pre	Post	¿Respeta?	Por qué
A	`x >= 0`	retorna positivo	✔ Sí	Pre **relajada** (acepta más casos); post igual
B	`x > 10`	retorna positivo	✘ No	Pre **endurecida**: un cliente que cumple `x > 0` (ej. 3) rompería
C	`x > 0`	retorna positivo y par	✔ Sí	Post **fortalecida** (promete más)
D	`x > 0`	retorna cualquier entero	✘ No	Post **debilitada**: promete menos que el padre

Autoevaluación: ¿qué tan firme estás?

Contestá primero **sin mirar** las respuestas (están al final de la guía). Si dudás en una, volvé al resumen.

- ☐ **1.** Enunciá el principio de sustitución de Liskov con tus palabras.
- ☐ **2.** ¿Cuáles son los tres tipos de polimorfismo del apunte?

☐ **3.** Dado `Padre p = new Hijo();` ¿cuál versión de un método sobreescrito se ejecuta y de qué atributo oculto se lee? ¿Por qué?

☐ **4.** ¿Qué es el enlace tardío y cuándo se resuelve?

☐ **5.** ¿Qué puede acceder a un miembro `protected`?

☐ **6.** Diferenciá precondición de postcondición y decí quién garantiza cada una.

☐ **7.** ¿Un método debe verificar su precondición? ¿Cuándo aparece una excepción en el contrato?

☐ **8.** Al sobreescribir, ¿se pueden endurecer las precondiciones? ¿y fortalecer las postcondiciones?

☐ **9.** Explicá la técnica del experto con un ejemplo del club.

☐ **10.** ¿Por qué `p.atributo3` no compila si `p` apunta a un Hijo que lo tiene?

Unidad 4 · Interfaces, doble envío y aserciones (Clase 5 y Proceso disciplinado)

1. Herencia simple vs herencia múltiple

Con **herencia simple** una clase nueva se define a partir de **una** clase; con **herencia múltiple**, a partir de varias. Java **no** tiene herencia múltiple de clases porque trae el **problema del diamante**: si B y C heredan de A y D hereda de B y C, y D invoca un método definido en A, ¿lo hereda por B o por C? El problema aparece al heredar **implementación**.

> **DEFINICIÓN** La solución de Java es la **interface**: permite heredar un **contrato abstracto sin heredar la implementación** (herencia de interfaz), con los beneficios de la herencia múltiple y sin sus problemas.

2. Interfaces

- La unidad fundamental en Java es la clase, pero la unidad fundamental del diseño OO es el **tipo**. Una interfaz define un **tipo de forma abstracta**, como una colección de métodos (el contrato).
- Es un conjunto de **declaraciones de métodos sin definición** y puede definir **constantes** (implícitamente `public static final`, siempre inicializadas).
- Se declara con `interface` y se implementa con `implements`. La clase debe implementar **todos** los métodos.
- Se puede declarar **variables** del tipo de la interfaz, pero **no instanciarla** (`new Interfaz()` no compila).
- Una instancia es del tipo de **su clase y de la interfaz** que implementa.
- Una interfaz puede **extender varias interfaces** (herencia múltiple de interfaces) y una clase puede **implementar varias**.
- Permite que objetos que **no comparten jerarquía** sean del mismo tipo por implementar la misma interfaz.

> **NOTA** La interfaz establece **qué** debe hacer la clase; la clase decide **cómo**. Implementar varias interfaces le da a una clase varios comportamientos (ej.: `Robot` implementa `Desplazable`, `Hablador`, `Calculador`).

Si declarás `Hablador robot = new Robot();`, del objeto solo ves lo que declara la interfaz `Hablador`: `robot.metodoX()` solo compila si `metodoX` está en `Hablador`.

¿Una interfaz puede tener atributos `final` no estáticos? (la inquietud del archivo "Intefaces")

En el documento "Intefaces", el docente nota que el IDE no marca error en `public final int peso = 40;` aunque no sea `static`, y sí lo marca en `public int edad;`. La bibliografía (Gosling, Eckel) es clara: **los campos de una interfaz son implícitamente `public static final`**. Entonces `final` sin `static` no declara un atributo de instancia: el campo es **igualmente estático** (el `static` va implícito), por eso no hay error. `int edad;` falla porque una constante de interfaz **debe inicializarse** en su declaración.

> **OJO** *Extra (versiones modernas de Java):* desde Java 8 las interfaces también pueden tener métodos `default` y `static` con implementación. Tu apunte describe la regla clásica ("todos los métodos son abstractos"); respondé con esa, y mencioná esto solo como aclaración.

3. Clase abstracta vs interfaz

	Clase abstracta	Interfaz
Implementación	Métodos ejecutables **y** abstractos	Sin código de implementación (todo abstracto)
Cuántas se pueden usar	Una clase **extiende una sola**	Una clase **implementa N** interfaces
Atributos y constructores	Variables de instancia, constructores y cualquier visibilidad	Sin variables de instancia ni constructores; solo constantes y métodos públicos
Relación con la clase	Hay **herencia** ("es un"); la subclase queda en la jerarquía	**No** hay herencia; la clase puede estar en cualquier lugar de la jerarquía
Se usa para...	Compartir código base y estado entre clases relacionadas	Definir un **comportamiento/contrato** que pueden cumplir clases no relacionadas

4. Principio de segregación de interfaces (ISP)

"Muchas interfaces específicas son mejores que una única más general" y **"los clientes no deberían verse forzados a depender de interfaces que no usan"**. Una interfaz "pesada" o contaminada obliga a implementar métodos que no se necesitan (quedan vacíos). La solución es **dividirla por funcionalidad** para que cada cliente implemente solo lo que usa.

5. Interfaces propias de Java: `Comparable`

`Comparable` declara un único método, `compareTo(Object o)`, que devuelve un valor **negativo, cero o positivo** según el receptor sea menor, igual o mayor que el parámetro. Cualquier clase que la implemente puede ordenarse con una clase ordenadora genérica (compará con el ejemplo `Ordenable` de la Clase 3, que era una clase abstracta).

6. Doble envío (double dispatch)

¿Cómo modelar una interacción cuando el resultado **depende del receptor y del argumento**? Por ejemplo, Piedra-Papel-Tijera. La solución *mal diseñada* es consultar la clase del parámetro con `if`/`switch`. La solución con doble envío: el método original **no pregunta nada**; simplemente le envía **otro mensaje al parámetro**, pasándose a sí mismo como argumento, y **cada objeto sabe qué hacer**.

- **Ventaja:** elimina los `if/switch` según la clase del parámetro y apoya la extensión **agregando** métodos en vez de modificarlos.
- **Inconveniente:** agregar una nueva clase de parámetro obliga a agregar un método secundario en las demás.

7. Proceso disciplinado: aserciones, pre/post e invariantes

Concepto	Qué afirma	Quién lo garantiza
Precondición	Requisitos para que la rutina funcione	El que invoca
Postcondición	El efecto/resultado de la rutina	El método invocado
Invariante de clase	Propiedades que **toda instancia cumple siempre** (valores válidos de atributos, restricciones sobre asociaciones, relaciones entre atributos)	Todos los métodos y constructores
Invariante de bucle	Condición verdadera **justo antes y justo después de cada iteración**. Si falla, el algoritmo "se perdió"	El ciclo

En todo método se puede suponer que **al comenzar** se cumplen el invariante de clase y la precondición; **al terminar** el objeto debe cumplir el invariante y la postcondición.

Aserciones en Java

- Una **aserción** es una condición lógica insertada en el código que se asume cierta; el sistema la comprueba y avisa con un `AssertionError` si es falsa.
- Sintaxis: `assert condicion;` o `assert condicion : "mensaje";`
- Por defecto están **desactivadas**; se activan con la opción **`-ea`** de la JVM (`java -ea MiClase`). Se aconsejan durante el **desarrollo**.
- `AssertionError` es un **Error**: no debería capturarse y el programa debería terminar.

Limitaciones de los asertos (el apunte las enumera)

- No permiten referirse al **valor antiguo** de un atributo (hace falta una variable local `oldSaldo`).
- No admiten cuantificadores lógicos ni condiciones **globales** a la clase (ej. "cada cuenta tiene un código único").
- Solo se comprueban **en ejecución**; **no se heredan**; **están desactivados por defecto**.

Aserciones	Excepciones
Verifican **suposiciones del programador** (corrección interna)	Notifican **errores en tiempo de ejecución** que el programa puede manejar
Pensadas para la etapa de desarrollo; se pueden desactivar	Siempre activas; forman parte del comportamiento
Fallan con `AssertionError` (no se captura)	Se capturan con `try/catch` o se propagan

Términos clave de la Unidad 4

Término	En una frase
Interfaz	Contrato abstracto: qué se debe hacer, no cómo
Problema del diamante	Ambigüedad al heredar la misma implementación por dos caminos
ISP	Interfaces chicas y específicas; nadie depende de lo que no usa
`Comparable`	`compareTo` devuelve <0, 0 o >0
Doble envío	Resolver según receptor y argumento enviando un segundo mensaje al argumento
Invariante de clase	Lo que debe ser cierto siempre para toda instancia
`assert` / `-ea`	Comprobación en desarrollo / opción para activarla

Ejercicio 4.1 · Piedra, Papel o Tijera con doble envío

Convención: `pierdeConX()` devuelve **true si el objeto receptor pierde contra X**. Tres jugadas, tres métodos: la matriz de resultados queda repartida *dentro de cada clase*, sin ningún `if`.

Paso 1. Definimos la interfaz `Jugada` con `leGanaA(Jugada otra)` y un método por cada jugada rival.

Paso 2. En cada clase, `leGanaA(otra)` **no pregunta de qué clase es `otra`**: le reenvía un segundo mensaje, pasándole su propia identidad en el nombre del método. En `Piedra`: `return otra.pierdeConPiedra();`.

Paso 3. Cada clase contesta según lo que sabe de sí misma. Por ejemplo, `Tijera.pierdeConPiedra()` es **true** (la tijera pierde con la piedra) y `Papel.pierdeConPiedra()` es **false**.

```
interface Jugada {
    boolean leGanaA(Jugada otra);
    boolean pierdeConPiedra();
    boolean pierdeConPapel();
    boolean pierdeConTijera();
}
class Piedra implements Jugada {
    public boolean leGanaA(Jugada otra) { return otra.pierdeConPiedra(); }
```

```
    public boolean pierdeConPiedra() { return false; }
    public boolean pierdeConPapel()  { return true;  }
    public boolean pierdeConTijera() { return false; }
}
class Papel implements Jugada {
    public boolean leGanaA(Jugada otra) { return otra.pierdeConPapel(); }
    public boolean pierdeConPiedra() { return false; }
    public boolean pierdeConPapel()  { return false; }
    public boolean pierdeConTijera() { return true;  }
}
class Tijera implements Jugada {
    public boolean leGanaA(Jugada otra) { return otra.pierdeConTijera(); }
    public boolean pierdeConPiedra() { return true;  }
    public boolean pierdeConPapel()  { return false; }
    public boolean pierdeConTijera() { return false; }
}
```

Salida real (verificada ejecutando el código):

```
Piedra le gana a Tijera: true
Piedra le gana a Papel: false
Tijera le gana a Papel: true
Piedra le gana a Piedra: false
```

Seguimiento de `piedra.leGanaA(tijera)`:

Paso 1. `piedra.leGanaA(tijera)` se ejecuta en `Piedra` (enlace tardío #1, por el receptor).

Paso 2. Ese método hace `tijera.pierdeConPiedra()` (enlace tardío #2, por el argumento: por eso se llama **doble** envío).

Paso 3. `Tijera.pierdeConPiedra()` devuelve **true** → la piedra le gana a la tijera. Un empate (piedra vs piedra) da **false**.

> **TIP DE PARCIAL** Si en el parcial te muestran un código con `if (otra instanceof Papel) ... else if (otra instanceof Tijera)...`, la respuesta de diseño que esperan es **doble envío**.

Ejercicio 4.2 · Segregar una interfaz "pesada" (ISP)

Una empresa define `interface Worker { void trabajar(); void descansar(); }` y ahora incorpora un **Robot** que trabaja pero no descansa. ¿Qué problema hay y cómo se resuelve?

Paso 1. Problema: `Robot` está obligado a implementar `descansar()`, un método que no usa (queda vacío o lanza un error). La interfaz **no es cohesiva**.

Paso 2. Solución: dividirla por funcionalidad en `Trabajador` y `Descansador`.

Paso 3. `Humano` implementa **ambas**; `Robot` implementa solo `Trabajador`. Cada cliente depende únicamente de lo que usa.

```
interface Trabajador  { void trabajar(); }
interface Descansador { void descansar(); }
class Humano implements Trabajador, Descansador {
    public void trabajar()  { System.out.println("Humano trabaja"); }
    public void descansar() { System.out.println("Humano descansa"); }
}
class Robot implements Trabajador {
    public void trabajar()  { System.out.println("Robot trabaja"); }
}
// main: Trabajador[] equipo = { new Humano(), new Robot() }; for (t : equipo) t.trabajar();
```

Salida real (verificada ejecutando el código):

```
Humano trabaja
Robot trabaja
```

Ejercicio 4.3 · Implementar `Comparable` para ordenar

Paso 1. La clase `Alumno` implementa `Comparable<Alumno>` y define `compareTo`.

Paso 2. Criterio elegido: comparar por legajo. Devolver negativo si `this` es menor, cero si son iguales, positivo si es mayor (`this.legajo - o.legajo`).

Paso 3. `Arrays.sort(...)` usa ese contrato sin saber nada de `Alumno`: solo conoce la interfaz `Comparable`.

```
class Alumno implements Comparable<Alumno> {
    int legajo; String nombre;
    Alumno(int l, String n) { legajo = l; nombre = n; }
    public int compareTo(Alumno o) { return this.legajo - o.legajo; }
    public String toString() { return legajo + "-" + nombre; }
}
// main: Alumno[] as = { new Alumno(30,"Zoe"), new Alumno(10,"Ana"), new Alumno(20,"Leo") };
//       java.util.Arrays.sort(as);  System.out.println(java.util.Arrays.toString(as));
```

Salida real (verificada ejecutando el código):

```
[10-Ana, 20-Leo, 30-Zoe]
```

OJO *Extra:* en el apunte `compareTo` recibe un `Object`. En Java moderno se usa la versión genérica `Comparable<Alumno>` (como acá) y se evita el cast.

Ejercicio 4.4 · Aserciones: pre, post e invariante de la clase Cuenta

Implementá `ingreso(double cantidad)` con: **pre** `cantidad >= 0`, **post** `saldo = saldoAnterior + cantidad` e **invariante** `titular != null` (más `saldo >= 0` en nuestra versión).

Paso 1. Las **precondiciones** van al comienzo: `assert cantidad >= 0 : "cantidad negativa";`.

Paso 2. Como un `assert` no puede mencionar el valor viejo del atributo, lo **guardamos en una variable local**: `double oldSaldo = saldo;`.

Paso 3. Se ejecuta la operación: `saldo = saldo + cantidad;`.

Paso 4. La **postcondición** se comprueba después: `assert saldo == oldSaldo + cantidad : "fallo postcondicion";`.

Paso 5. El **invariante** se concentra en un método privado `invariante()` que se llama al final de cada método y constructor.

```
class Cuenta {
    private double saldo;  private String titular;
    public Cuenta(String titular, double saldoInicial) {
        assert titular != null : "titular nulo";
        this.titular = titular;  this.saldo = saldoInicial;
        assert invariante() : "Fallo invariante";
    }
    public void ingreso(double cantidad) {
        assert cantidad >= 0 : "cantidad negativa";
        double oldSaldo = saldo;
        saldo = saldo + cantidad;
        assert saldo == oldSaldo + cantidad : "fallo postcondicion";
        assert invariante() : "Fallo invariante";
    }
    private boolean invariante() { return titular != null && saldo >= 0; }
}
// main: c = new Cuenta("Ana", 100); c.ingreso(50); c.ingreso(-20);
```

Salida real (verificada ejecutando el código):

```
--- java Main   (aserciones DESACTIVADAS, el comportamiento por defecto)
saldo = 150.0
saldo = 130.0          <- ¡se aceptó un ingreso negativo!
--- java -ea Main  (aserciones ACTIVADAS)
saldo = 150.0
Exception in thread "main" java.lang.AssertionError: cantidad negativa
    at Cuenta.ingreso(Main.java:11)
```

TRAMPA CLÁSICA Sin `-ea` las aserciones **ni se evalúan**. Por eso **nunca** se usan para validar datos que vienen del usuario: para eso están las excepciones.

Ejercicio 4.5 · Identificar pre, post e invariante + invariante de bucle

Clase Cuenta del apunte. Clasificá cada enunciado:

Enunciado	Tipo
El estado de una cuenta es correcto si tiene un titular (referencia no nula)	**Invariante** de clase
`ingreso()` incrementa el saldo de la cuenta	**Postcondición**
El parámetro de `ingreso()` debe ser siempre positivo	**Precondición**
`ingreso()` solo se puede ejecutar en estado OPERATIVA	**Precondición** (sobre el estado del objeto)
Cada cuenta tiene un código diferente	**Invariante global** (no expresable con un `assert` simple)

Invariante de bucle (búsqueda binaria en un diccionario): en toda iteración debe ser cierto que *la palabra buscada, si existe, está dentro de la porción que todavía estamos examinando* (es posterior a la primera palabra de la porción y anterior a la última). Si esto deja de ser cierto, el algoritmo se perdió.

Autoevaluación: ¿qué tan firme estás?

Contestá primero **sin mirar** las respuestas (están al final de la guía). Si dudás en una, volvé al resumen.

- ☐ **1.** ¿Qué es el problema del diamante y cómo lo evita Java?
- ☐ **2.** Enumerá tres diferencias entre clase abstracta e interfaz.
- ☐ **3.** ¿Se puede instanciar una interfaz? ¿Se puede declarar una variable de su tipo?
- ☐ **4.** ¿Por qué `public final int peso = 40;` compila en una interfaz sin `static`?
- ☐ **5.** Enunciá el principio de segregación de interfaces y dá un ejemplo.
- ☐ **6.** ¿Qué problema resuelve el doble envío y qué ventaja y desventaja tiene?
- ☐ **7.** Diferenciá invariante de clase de invariante de bucle.
- ☐ **8.** ¿Cómo se activan las aserciones y qué ocurre cuando una falla?
- ☐ **9.** Nombrá tres limitaciones de los asertos de Java.
- ☐ **10.** ¿Cuál es el método de `Comparable` y qué significa su resultado?

Unidad 5 · Excepciones y clonación (Clase 6 y Clonación)

1. ¿Por qué excepciones?

Imaginá `int dividir(int a, int b) { return a / b; }` invocado con `b = 0`. ¿Qué hacer? Ni "poner un cartelito" ni devolver un valor "bandera" son buenas ideas: el problema **no se puede resolver dentro del método** y la verificación no debe ensuciar el flujo normal. Java usa **excepciones**.

> **DEFINICIÓN** Una **excepción** es un evento que ocurre durante la ejecución y **interrumpe el flujo normal** de las sentencias. Las excepciones son **objetos**.

Ventajas frente al manejo tradicional de errores:

- **Separan** el manejo de errores del código "normal" (código más limpio).
- **Propagan** el error por la pila de llamadas hasta quien sabe resolverlo.
- **Agrupan y diferencian** tipos de error (cada tipo es una clase).
- Convierten las condiciones de error en **parte explícita del contrato** del método (lista `throws`), que el compilador comprueba.

2. Jerarquía de excepciones

Clase	Qué representa	¿Hay que declararla en `throws`?
`Throwable`	Raíz: todo lo que se puede "lanzar"	—
`Error`	Problemas graves de la JVM (memoria agotada); la aplicación no debería tratar de resolverlos	No
`Exception`	Situaciones que una aplicación debería tratar razonablemente	**Sí** (son *comprobadas*), salvo `RuntimeException`
`RuntimeException`	Errores del programador: división por cero, índice fuera de rango, `null`	No (*no comprobadas*)
`IOException`	Errores que el programador no puede evitar (entrada/salida)	Sí

Por convención, las excepciones propias extienden `Exception`. **`RuntimeException` y `Error` son las únicas que no hace falta incluir en la cláusula `throws`**: cualquier método puede lanzarlas.

3. try / catch / finally / throw / throws

Palabra	Rol
`try`	Zona de ejecución **normal**. Si una sentencia lanza una excepción, las siguientes del bloque **no se ejecutan**
`catch (Tipo e)`	Zona de **recuperación**: recibe el objeto excepción y elabora la solución
`finally`	Zona de **finalización**: se ejecuta siempre que el `try` haya comenzado
`throw`	**Lanza** un objeto excepción (`throw new MiExcepcion(...)`) ← es una sentencia
`throws`	En la cabecera: **declara** las excepciones comprobadas que el método **propaga** ← es una cláusula

Reglas del catch

- La excepción lanzada "revisa" los `catch` en orden y la atrapa **el primero** cuyo tipo coincide (o es padre de su tipo).

- Por eso hay que poner **primero las excepciones hijas y después las padre**; al revés, el compilador marca error.
- Si ningún `catch` la atrapa, **se propaga** al método invocante (el método debe declararla en `throws`, si es comprobada).

Las 3 opciones al invocar un método que declara una excepción comprobada

Paso 1. Capturarla y gestionarla con `catch`.

Paso 2. Capturarla y transformarla en otra excepción propia (`throw new Otra()`), que debe estar en tu `throws`.

Paso 3. **Declararla en tu `throws`** y dejar que pase de largo (propagar).

`throws` y redefinición de métodos

Un método que redefine o implementa **no puede declarar más excepciones comprobadas** que el del padre. Sí puede lanzar **subtipos** de las declaradas (se capturan con el `catch` del supertipo). Si un método no tiene `throws`, no significa que pueda lanzar cualquiera: **no puede lanzar excepciones comprobadas**. Los constructores sí pueden declarar y lanzar comprobadas.

4. `finally` bien usado

- Si el `try` comenzó, `finally` **siempre se ejecuta**: después del `try` sin excepción, después de un `catch` que la atrapó, o justo antes de propagar una excepción no capturada.
- Si hay `return` dentro del `try`, `finally` igualmente se ejecuta **antes** de retornar.
- Si el `finally` tiene su propio `return`, **gana ese** y se descarta el del `try` (y cualquier excepción pendiente).
- **Patrón pre/post:** si `pre()` debe ir seguido de `post()` pase lo que pase, escribí `pre()` **fuera** del `try` y `post()` en el `finally`. Así, si `pre()` falla, `post()` no se ejecuta.

5. Crear excepciones propias

Una excepción propia extiende `Exception`, llama a `super("mensaje para el programador")` y **guarda los datos** que servirán en la zona de recuperación (con un getter). Al lanzarla se crea con `new` y se le pasan esos datos; el `catch` los recupera con `e.getDato()`.

> **TIP DE PARCIAL** Cuándo usar cada cosa (síntesis de la Unidad 3 y 5): **precondición** = lo que quien llama debe garantizar (no se verifica); **excepción** = cuando, aun cumpliéndose la precondición, no se puede cumplir la postcondición; **aserción** = control interno durante el desarrollo.

6. Comparaciones imprescindibles

	Comprobadas (checked)	No comprobadas (unchecked)
Clases	`Exception` y subclases (menos `RuntimeException`)	`RuntimeException`, `Error` y sus subclases
`throws`	Obligatorio declarar o capturar	No hace falta
Origen típico	Situaciones externas (E/S, recursos)	Errores de programación
Ejemplos	`IOException`, excepciones propias que extienden `Exception`	`ArithmeticException`, `NullPointerException`, `IllegalArgumentException`

`throw`	`throws`
Sentencia: **lanza** un objeto excepción	Cláusula de la cabecera: **declara** las comprobadas que el método puede propagar
`throw new Mi_Exception();`	`void metodo() throws Mi_Exception`

Pregunta del apunte: `final` vs `finally` vs `finalize`

Palabra	Qué es	Dónde se usa
`final`	**Modificador**: constante / método no redefinible / clase no extensible	Variables, métodos, clases
`finally`	**Bloque** que siempre se ejecuta tras un `try`	Manejo de excepciones
`finalize()`	**Método** que llama el recolector antes de liberar un objeto	Liberar recursos no Java

7. Clonación: `clone()`

Clonar es duplicar un objeto: quedan dos objetos con la **misma información** pero **identidad distinta**; los cambios posteriores en uno no afectan al otro. `Object.clone()` es `native` y `protected`: reserva memoria equivalente y **copia campo por campo** el estado original al nuevo. Devuelve un `Object` (hace falta un cast).

El problema de la copia superficial (shallow)

Por defecto, para atributos **primitivos** se copia el valor; para atributos **objeto** se copia **la referencia**. Resultado: original y clon comparten los mismos objetos internos (por ejemplo, el mismo arreglo, porque *los arreglos son objetos*). Modificar uno modifica al otro.

	Copia superficial (shallow)	Copia profunda (deep)
Atributos primitivos	Se copia el valor	Se copia el valor
Atributos objeto	Se copia la **referencia** (se comparten)	Se **clona** también cada objeto interno
Cómo se logra	`return super.clone();`	`super.clone()` + clonar a mano los campos objeto
Riesgo	Efectos colaterales entre original y clon	Más trabajo; los componentes deben ser clonables

Receta correcta de `clone()`

Paso 1. La clase implementa `Cloneable`.

Paso 2. Primero invoca `super.clone()` (así crea un objeto del **tipo correcto**, incluso para subclases; usar `new` rompería a las subclases).

Paso 3. Después "arregla" los campos donde copiar el valor es incorrecto (ej. `buffer = (int[]) buffer.clone();`).

Paso 4. Atrapa `CloneNotSupportedException` (excepción comprobada) y la convierte, p. ej., en un `InternalError`.

Tres actitudes frente a `clone()`

Actitud	Cómo se implementa
Admitir clone	Implementa `Cloneable` y declara `clone()` sin lanzar excepciones
Admitir **condicionalmente**	Implementa `Cloneable` pero deja pasar la `CloneNotSupportedException` de los componentes que intenta clonar
Prohibir clone	No implementa `Cloneable` y su `clone()` lanza siempre `CloneNotSupportedException`

Las subclases que implementen `clone()` **no pueden lanzar** `CloneNotSupportedException` si el padre la eliminó de su declaración, porque un método redefinido no puede agregar excepciones.

Términos clave de la Unidad 5

Término	En una frase
Excepción	Objeto que representa un evento que interrumpe el flujo normal
Comprobada / no comprobada	Con / sin obligación de declarar o capturar
Propagación	La excepción sube por la pila hasta que un `catch` la atrapa
`finally`	Se ejecuta siempre que el `try` haya comenzado
Copia superficial	Clona el objeto pero comparte los objetos internos
`Cloneable`	Interfaz marcadora que habilita `clone()`
`CloneNotSupportedException`	Señala que no se debió llamar a `clone()` en esa clase

Ejercicio 5.1 · Trazar `try / catch / finally` con `return`

```
static int f() {
    try {
        System.out.println("A");
        throw new IllegalStateException("x");
    } catch (IllegalStateException e) {
        System.out.println("B");
        return 1;
    } finally {
        System.out.println("C");
    }
}
static int g() {
    try { return 1; } finally { return 2; }
}
static void a() throws Exception {
    try { b(); System.out.println("a-despues"); }
    finally { System.out.println("a-finally"); }
}
static void b() throws Exception { throw new Exception("boom"); }
// main: println(f()); println(g()); try { a(); } catch (Exception e) { println("main: " + e.getMessage()); }
```

Salida real (verificada ejecutando el código):

```
A
B
C
1
2
a-finally
main: boom
```

Paso 1. **`f()`:** imprime A; se lanza la excepción; el `catch` la atrapa e imprime B; el `return 1` queda "pendiente"; **antes de salir se ejecuta `finally`** (imprime C); recién ahí retorna 1. Por eso se ve A, B, C y después el 1.

Paso 2. **`g()`:** el `try` quiere retornar 1, pero el `finally` tiene su propio `return 2` y **gana este último**. Imprime 2.

Paso 3. **`a()`:** `b()` lanza `Exception`. En `a`, nada la captura, entonces `finally` se ejecuta **justo antes de propagar** (imprime a-finally) y `"a-despues"` **nunca** se imprime.

Paso 4. La excepción sube a `main`, que la atrapa y muestra su mensaje.

Otro caso del examen (verificado): ¿qué devuelve `t(n)` para n = 0, 1 y 2?

```
static String t(int n) {
    StringBuilder sb = new StringBuilder();
    try {
        sb.append("1");
        if (n == 1) throw new RuntimeException("r");
        sb.append("2");
        if (n == 2) return sb.append("R").toString();
    } catch (RuntimeException e) { sb.append("3"); }
    finally { sb.append("4"); }
```

```
    sb.append("5");
    return sb.toString();
}
```

Salida real (verificada ejecutando el código):

```
1245     (n=0)
1345     (n=1)
12R      (n=2)
```

Paso 1. n = 0: sin excepción: 1, 2, luego `finally` agrega 4 y, al salir del bloque, 5 → `1245`.

Paso 2. n = 1: tras "1" se lanza; el `catch` agrega 3; `finally` agrega 4; sigue con 5 → `1345`.

Paso 3. n = 2: el `return` **ya calculó** el String `"12R"` antes de entrar al `finally`. El `finally` agrega 4 al `StringBuilder`, pero el valor retornado ya se evaluó → **`12R`**.

Ejercicio 5.2 · Excepción propia con datos + orden de los catch

```
class NoExisteException extends Exception {
    private int legajo;
    public NoExisteException(int legajo) {
        super("No existe el legajo " + legajo);   // mensaje para el programador
        this.legajo = legajo;                      // dato para la recuperación
    }
    public int getLegajo() { return legajo; }
}
class Curso {
    private java.util.Map<Integer, String> alumnos = new java.util.HashMap<>();
    // ... el constructor carga los legajos 1 y 2
    public void actualizar(int legajo, String nuevo) throws NoExisteException {
        if (!alumnos.containsKey(legajo)) throw new NoExisteException(legajo);
        alumnos.put(legajo, nuevo);
    }
}
// main:
try { c.actualizar(1, "Ana Maria"); c.actualizar(99, "X"); println("no llega"); }
catch (NoExisteException e) { println("Error: " + e.getMessage() + " (legajo " + e.getLegajo() + ")"); }
finally { println("fin"); }
```

Salida real (verificada ejecutando el código):

```
Error: No existe el legajo 99 (legajo 99)
fin
```

Paso 1. `actualizar(1, ...)` funciona. `actualizar(99, ...)` lanza `NoExisteException` con el legajo.

Paso 2. Se **abandona** el resto del `try`: `"no llega"` jamás se imprime.

Paso 3. El `catch` recibe el objeto y usa `getLegajo()` para armar la respuesta. Después corre `finally` y se imprime "fin".

¿Y si invertimos el orden de los catch?

```
try { leer(); }                                  // leer() declara IOException
catch (Exception e)   { println("general"); }
catch (IOException e) { println("especifica"); }
```

Mensaje real del compilador:

```
Main.java:7: error: exception IOException has already been caught
        catch (IOException e) { System.out.println("especifica"); }
```

`Exception` es **padre** de `IOException`: el primer `catch` ya atrapa todo, así que el segundo es inalcanzable. **Siempre primero los hijos.**

Ejercicio 5.3 · Copia superficial vs profunda de una pila

```
// Versión 1: copia superficial
class PilaShallow implements Cloneable {
    protected int[] buf;  protected int tope = -1;
    @Override public Object clone() {
        try { return super.clone(); }
        catch (CloneNotSupportedException e) { throw new InternalError(e.toString()); }
    }
}
// Versión 2: copia profunda
class PilaDeep implements Cloneable {
    private int[] buf;  private int tope = -1;
    @Override public Object clone() {
        try {
            PilaDeep nObj = (PilaDeep) super.clone();
            nObj.buf = (int[]) buf.clone();          // <- clona también el arreglo
            return nObj;
        } catch (CloneNotSupportedException e) { throw new InternalError(e.toString()); }
    }
}
// Prueba (en ambas): a.meter(2); a.meter(5); b = a.clone(); b.sacar(); b.meter(7); a.sacar()
```

Salida real (verificada ejecutando el código):

```
shallow: a.sacar() = 7
deep:    c.sacar() = 5
```

Paso 1. Se arma la pila original con los valores 2 y 5 (el tope es 5) y se clona.

Paso 2. Shallow: `super.clone()` copia el campo `buf` como **referencia**: original y clon usan **el mismo arreglo**. Cuando el clon hace `sacar()` y `meter(7)`, sobrescribe el 5 por el 7 *en el arreglo compartido*. Al hacer `sacar()` en el original aparece **7** en lugar de 5. Se rompió la independencia.

Paso 3. Deep: además de `super.clone()`, se clona el arreglo (`buf.clone()`), así cada pila tiene su propio arreglo. El original conserva su 5.

Paso 4. Moraleja: tras `super.clone()` hay que revisar **cada atributo objeto** y decidir si copiarlo.

Ejercicio 5.4 · ¿Qué tipo de excepción usar?

Situación	Decisión
`dividir(a, b)` recibe `b = 0` y la precondición dice `b != 0`	No verificar: es culpa de quien llama; en la práctica salta `ArithmeticException` (no comprobada)
`actualizar(legajo)` con legajo inexistente aunque se cumplan las pre	Excepción **comprobada propia** (`NoExisteException`) declarada en `throws` y documentada en el contrato
`new FileReader("x")` y el archivo no existe	Excepción comprobada (`IOException`): hay que capturarla o declararla
Se acaba la memoria de la JVM	`Error`: no se intenta recuperar

Autoevaluación: ¿qué tan firme estás?

Contestá primero **sin mirar** las respuestas (están al final de la guía). Si dudás en una, volvé al resumen.

- ☐ **1.** Diferenciá `Error`, `Exception` y `RuntimeException`.
- ☐ **2.** ¿Qué excepciones no hace falta declarar en `throws`?
- ☐ **3.** Un método invoca a otro que declara una excepción comprobada. ¿Qué tres opciones tiene?
- ☐ **4.** ¿Por qué hay que ubicar los `catch` de hijas antes que los de padres?
- ☐ **5.** ¿En qué casos se ejecuta `finally`? ¿Qué pasa si tiene un `return`?
- ☐ **6.** Un método redefinido, ¿puede lanzar más excepciones comprobadas que el del padre?
- ☐ **7.** Diferenciá `throw` de `throws`.
- ☐ **8.** Explicá `final`, `finally` y `finalize`.

☐ **9.** ¿Qué es una copia superficial y por qué `Object.clone()` por defecto puede ser peligroso?

☐ **10.** ¿Por qué `clone()` debe empezar con `super.clone()` y no con `new`?

☐ **11.** Nombrá las tres actitudes frente a `clone()`.

Unidad 6 · Patrones de diseño y frameworks

Un patrón es una **solución probada** a un problema que se repite. Esta unidad se estudia mejor con una pregunta fija para cada patrón: **¿qué problema resuelve?, ¿cómo se arma? y ¿cómo lo reconozco en el enunciado?**

1. ¿Qué es un patrón de diseño?

> **DEFINICIÓN** "Cada patrón describe un problema que ocurre una y otra vez en nuestro entorno y describe también el núcleo de la solución, de forma que puede utilizarse un millón de veces sin tener que hacer dos veces lo mismo." (Alexander). Para Gamma: **descripción de clases y objetos comunicándose entre sí, adaptada para resolver un problema de diseño general en un contexto particular.**

El objetivo es la **reutilización**: los diseñadores expertos no resuelven todo desde cero, reutilizan soluciones que funcionaron.

Lo que SÍ vienen "gratis" con un patrón	Lo que NO viene gratis
Experiencia acumulada	La **creatividad**: un patrón no garantiza nada por sí solo
Un **vocabulario común** entre desarrolladores	Reemplazar al humano en el proceso creativo
Documentación más fácil de leer	Aplicarlo correctamente sin entender el problema
Facilita el rediseño	

Los patrones se clasifican según su utilidad: **creacionales** (cómo se crean objetos), **de comportamiento** (cómo colaboran y reparten responsabilidades) y **estructurales** (cómo se componen objetos y clases).

Resumen rápido de los seis patrones

Patrón	Tipo	Problema que resuelve	Pista en el enunciado
Singleton	Creacional	Debe existir **una única instancia** con acceso global	"un único", "una sola instancia", gestor, cola de impresión
Factory	Creacional	Crear objetos **sin exponer la lógica de creación**, sin saber la clase concreta	"según un parámetro se crea…", "no sabemos qué objeto"
Template Method	Comportamiento	Algoritmo con **pasos fijos** y algunos que varían por subclase	"mismos pasos en el mismo orden", "solo cambia un paso"
Decorator	Estructural	Agregar responsabilidades **dinámicamente** a un objeto, evitando la explosión de clases	"extras opcionales", "combinaciones", "agregar sin modificar"
Observer	Comportamiento	Notificar a **muchos dependientes** cuando cambia un objeto, con bajo acoplamiento	"cuando cambie… se actualicen todos", "suscriptores"
State	Comportamiento	El comportamiento **depende del estado interno**; evitar `if` por estado	"según el estado se comporta distinto", transiciones

La clasificación de Singleton y Decorator como creacional y estructural viene de la bibliografía clásica (GoF); tus apuntes marcan explícitamente a Factory como creacional y a Template y State como de comportamiento.

2. Singleton: "un único objeto siempre"

Objetivo: asegurar que una clase tenga **una única instancia** y darle acceso global. Útil para el acceso a un sistema de archivos, la cola de impresión o el gestor de ventanas. **La propia clase controla** que la instancia sea única.

- Un **constructor privado**: nadie puede hacer `new`.
- Un **atributo estático privado** del propio tipo que guarda la instancia.
- Un **método estático** `getInstance()`: si el atributo es `null` crea el objeto; si no, devuelve el existente.

Falta de…	Consecuencia
Constructor privado	Cualquiera crea más instancias con `new`
Atributo estático	No hay dónde guardar la única instancia
Método estático de acceso	No hay forma de obtener la instancia desde afuera

3. Factory: encapsular la creación

Patrón **creacional**. Se **crea un objeto sin exponer la lógica de creación** al cliente y se devuelve una **referencia mediante una interfaz común**. En lugar de `new Circle()`, el cliente pide `ShapeFactory.getShape("CIRCLE")`. Se apoya en el **polimorfismo**: el cliente trabaja con el tipo `Shape`, sin saber qué clase concreta se instanció. Objetivo principal: **encapsular la creación de objetos**.

4. Template Method

Objetivo: permitir que ciertos **pasos** de un algoritmo se redefinan en las subclases **sin cambiar el algoritmo completo**. La clase base (abstracta) define la **plantilla**: pasos comunes implementados, pasos que dependen del caso declarados **abstractos**, y el orden del algoritmo (**las hijas no pueden alterarlo**).

- **Inversión de dependencias:** la clase padre es quien llama a los métodos de las hijas, sin conocerlas.
- **Encapsular lo que cambia**: lo variable va en un método aparte.
- **Hooks (ganchos):** métodos con **comportamiento por defecto** en la clase base, que las derivadas pueden sobrescribir o no. Dan pasos **opcionales** (muchas veces con una condición).
- **Cuándo usarlo:** algoritmo igual con pasos distintos; el "qué" está definido y el "cómo" varía; mucho código duplicado que se puede factorizar.

Lección del ejemplo del motor de 4 tiempos: sin hooks el algoritmo es *rígido* (no se pueden añadir pasos intermedios como un turbo); con hooks se gana flexibilidad sin crear clases `MotorTurboGasolina`, `MotorTurboDiesel`…

5. Decorator

Objetivo: añadir responsabilidades a un objeto **de forma dinámica**; es una alternativa flexible a la herencia. Evita la **explosión de clases** (2 vehículos × 5 motores = 10 clases hoja).

Participante	Rol
Component	Define la interfaz de los objetos a los que se les puede añadir responsabilidades
ComponenteConcreto	El objeto "base" al que se le agregan responsabilidades (ej. `Berlina`)
Decorator	Hereda del mismo tipo que el componente y **mantiene una referencia** a otro Component (recibida por constructor)
DecoratorConcreto	Agrega la responsabilidad: hace algo antes y/o después de **delegar** en el objeto envuelto (ej. `Turbo`, `Diesel`)

- El decorador **hereda del mismo tipo** no para reutilizar código, sino para que sea **intercambiable** (Liskov): un decorador puede reemplazar al objeto decorado.
- Se pueden **encapsular iterativamente** (decorador de decorador); el cliente interactúa con la **capa más externa**.
- También se lo llama **Wrapper**. Resumen: **sustituye la herencia por un proceso iterativo de composición.**

- Consecuencias: más flexible que la herencia, evita clases altas sobrecargadas; pero **componente y decorador no son el mismo objeto**, genera muchos objetos pequeños (depuración difícil) y permite combinaciones inválidas o referencias circulares.

6. Observer–Observable (Publish–Subscribe)

Intención: definir una dependencia entre objetos de modo que **cuando un objeto cambia su estado, todos sus dependientes son notificados** y actualizados automáticamente, **sin aumentar el acoplamiento**. Ejemplo típico: separar la presentación de los datos.

Participante	Responsabilidades
Observable (abstracta / clase `Observable` de Java)	Conoce a sus observadores (cualquier cantidad); permite agregarlos y quitarlos; guarda el estado de interés; **notifica** cuando cambia
Observer (interfaz)	Define la interfaz de los objetos a notificar: un único método `update(Observable o, Object arg)`
Observado concreto	Almacena el estado de interés y envía las notificaciones
Observador concreto	Mantiene referencia al observado, guarda lo que le interesa e implementa `update` para mantenerse consistente

En Java

- El observable **extiende** `Observable`; el observador **implementa** `Observer`.
- Para avisar: `setChanged();` y luego `notifyObservers(arg)`. `notifyObservers` ejecuta `update` de cada observador y limpia la marca de "modificado".
- Otros métodos: `addObserver`, `deleteObserver`, `deleteObservers`, `countObservers`, `hasChanged`.

Consecuencias	Detalle
✔ Bajo acoplamiento abstracto	El sujeto no conoce la clase concreta de sus observadores
✔ Se pueden agregar observadores sin tocar el observado	Y reutilizar sujeto y observadores por separado
✔ Broadcast	Notifica a todos los suscritos
✘ Actualizaciones inesperadas	Una operación puede disparar una cascada de cambios; el protocolo no dice qué cambió

Cuestión de implementación: ¿quién llama a `notify`? Si lo hace el **sujeto** al cambiar su estado, los clientes no hacen nada pero puede ser ineficiente con varios cambios seguidos; si lo hacen los **clientes**, se puede optimizar pero tienen más responsabilidad.

> **OJO** *Extra:* `java.util.Observable` y `Observer` están **deprecadas desde Java 9**, pero siguen funcionando y son las que usa tu apunte; el concepto del patrón es el mismo.

7. State

Objetivo: permitir que un objeto **modifique su comportamiento cuando cambia su estado interno**; parecerá que cambia de clase. Encapsula el comportamiento según el estado. **Problema:** si cada método pregunta con `if` por el estado, el código se llena de condicionales. **Solución:** *encapsular el estado*.

Paso 1. Se crea una **interfaz de estado** común (`Estado`) con los métodos que dependen del estado (acelerar, frenar, contacto).

Paso 2. Se crea una **clase de contexto** (`Vehiculo`) que guarda el **estado actual** y delega en él cada mensaje.

Paso 3. Se crea **una clase por estado** que implementa la interfaz; cada método implementa la **transición**: cambia el estado del contexto seteando uno nuevo (o no hace nada).

Paso 4. Cada estado recibe en su constructor la **referencia al contexto**, para poder hacer la transición.

> **TIP DE PARCIAL** Diferencia clave entre **State** y **Template Method**: en State el comportamiento cambia **en tiempo de ejecución** según el estado (composición + transiciones); en Template el algoritmo es fijo y varía por **subclase** (herencia).

8. Frameworks

Los objetos suelen ser demasiado chicos y específicos para reutilizarlos entre sistemas; los **frameworks** son abstracciones de grano más grueso. Un **framework** es una **estructura genérica** (conjunto integrado de clases abstractas y concretas, objetos y componentes) que se **extiende** para crear una aplicación o subsistema específico. Ofrece reutilización de **diseño** (la arquitectura es el esqueleto) y de clases concretas.

Tipo de framework	Qué soporta	Ejemplos
De infraestructura de sistema	Comunicaciones, interfaces de usuario, compiladores	—
De integración de middleware	Comunicación de componentes e intercambio de información	.NET, Enterprise Java Beans
De aplicación empresarial	Dominios específicos (telecomunicaciones, finanzas)	—
WAF (aplicación web)	Seguridad, páginas dinámicas, BD, sesiones. Suelen basarse en **MVC**	Spring, Hibernate (citados)

Características notables

- **Inversión de control:** a diferencia de una biblioteca, el flujo lo dicta **el framework**, no el programa que llama.
- **Comportamiento predeterminado:** útil, definido e identificable.
- **Extensibilidad:** se amplía (por herencia, hooks, etc.) pero **no se modifica su código**.
- Se construyen con **patrones** y tienen **alta cohesión y bajo acoplamiento**. Los **objetos calientes** vinculan las necesidades del sistema con la funcionalidad; los **fríos** casi no cambian y dan portabilidad.

Biblioteca / API	Framework
Tu código **la llama**	El framework **llama a tu código** (inversión de control)
Funcionalidad dentro de métodos	Funcionalidad a nivel de grupo de objetos / arquitectura
Se usa	Se **extiende**

Términos clave de la Unidad 6

Término	En una frase
Patrón de diseño	Solución reutilizable a un problema de diseño recurrente
Creacional / comportamiento	Cómo se crean los objetos / cómo colaboran y reparten tareas
Singleton	Una única instancia con acceso global
Factory	Crear objetos sin exponer la lógica de creación
Template Method / hook	Algoritmo fijo con pasos variables / paso opcional con comportamiento por defecto
Decorator (wrapper)	Añadir responsabilidades dinámicamente envolviendo un objeto
Observer / `update`	Dependientes notificados automáticamente al cambiar el sujeto
State	Una clase por estado; el contexto delega en el estado actual
Framework / inversión de control	Esqueleto reutilizable y extensible / el framework llama al código del usuario

Ejercicio 6.1 · Singleton y Factory (verificados)

```
class Config {
    private static Config instancia;
    private Config() { System.out.println("Creando Config..."); }
    public static Config getInstance() {
        if (instancia == null) instancia = new Config();
        return instancia;
    }
}
interface Shape { void draw(); }
class Circle implements Shape { public void draw() { System.out.println("Dentro de Circle::draw()"); } }
class Square implements Shape { public void draw() { System.out.println("Dentro de Square::draw()"); } }
class ShapeFactory {
    public static Shape getShape(String tipo) {
        if (tipo == null) return null;
        if (tipo.equalsIgnoreCase("CIRCLE")) return new Circle();
        if (tipo.equalsIgnoreCase("SQUARE")) return new Square();
        return null;
    }
}
// main: c1 = Config.getInstance(); c2 = Config.getInstance(); println(c1 == c2);
//       f1 = ShapeFactory.getShape("CIRCLE"); f2 = ShapeFactory.getShape("square"); f1.draw(); f2.draw();
```

Salida real (verificada ejecutando el código):

```
Creando Config...
true
Dentro de Circle::draw()
Dentro de Square::draw()
```

Paso 1. Singleton: el primer `getInstance()` ve `instancia == null` y crea el objeto (se imprime "Creando Config..." **una sola vez**). El segundo la encuentra ya creada y la devuelve.

Paso 2. `c1 == c2` da **true**: son literalmente el mismo objeto. No se puede hacer `new Config()` desde afuera porque el constructor es privado.

Paso 3. Factory: el cliente solo conoce `Shape` y el nombre del tipo. `getShape("square")` devuelve un `Square` (ignora mayúsculas por `equalsIgnoreCase`). Agregar una figura nueva toca la fábrica, no al cliente.

Ejercicio 6.2 · Template Method con hook: el motor

```
abstract class Motor {
    public final void realizarCiclo() {          // PLANTILLA: orden fijo (final)
        preAdmision();                            // <- HOOK (opcional)
        admision();
        compresion();
        combustion();                             // <- abstracto: depende del motor
        escape();
    }
    protected void preAdmision() { }              // hook con comportamiento por defecto: nada
    protected void admision()  { System.out.println("Admision"); }
    protected void compresion(){ System.out.println("Compresion"); }
    protected abstract void combustion();
    protected void escape()    { System.out.println("Escape"); }
}
class MotorNafta extends Motor {
    protected void combustion() { System.out.println("Explosion por chispa"); }
}
class MotorDieselTurbo extends Motor {
    @Override protected void preAdmision() { System.out.println("Turbo comprime el aire"); }
    protected void combustion() { System.out.println("Combustion por presion"); }
}
```

Salida (los textos "---" son separadores que agregué para distinguir las dos ejecuciones):

```
--- new MotorNafta().realizarCiclo()
Admision
Compresion
Explosion por chispa
Escape
--- new MotorDieselTurbo().realizarCiclo()
Turbo comprime el aire
Admision
Compresion
Combustion por presion
Escape
```

Paso 1. `realizarCiclo()` es `final`: **las hijas no pueden cambiar el orden** del algoritmo.

Paso 2. Lo que **siempre cambia** (`combustion`) es abstracto: las hijas están **obligadas** a implementarlo.

Paso 3. `preAdmision()` es un **hook**: por defecto no hace nada. `MotorNafta` no lo redefine y no "se engancha". `MotorDieselTurbo` sí, y agrega el turbo **sin** crear clases nuevas por cada combinación.

Paso 4. Observá la **inversión de dependencias**: la clase `Motor` llama a métodos de sus hijas sin conocerlas.

Ejercicio 6.3 · Decorator: vehículos con motores (evitar la explosión de clases)

```
abstract class Vehiculo {                       // Component
    public abstract String descripcion();
    public abstract int precio();
}
class Berlina extends Vehiculo {                // ComponenteConcreto
    public String descripcion() { return "Berlina"; }
    public int precio() { return 10000; }
}
abstract class Decorador extends Vehiculo {     // Decorator: mismo tipo + referencia
    protected Vehiculo decorado;
    public Decorador(Vehiculo v) { this.decorado = v; }
}
class Diesel extends Decorador {                // DecoratorConcreto
    public Diesel(Vehiculo v) { super(v); }
    public String descripcion() { return decorado.descripcion() + " + Diesel"; }
    public int precio() { return decorado.precio() + 1500; }
}
class Turbo extends Decorador {
    public Turbo(Vehiculo v) { super(v); }
    public String descripcion() { return decorado.descripcion() + " + Turbo"; }
    public int precio() { return decorado.precio() + 800; }
}
// main: Vehiculo v = new Turbo(new Diesel(new Berlina()));  Vehiculo w = new Diesel(new Berlina());
```

Salida real (verificada ejecutando el código):

```
Berlina + Diesel + Turbo = 12300
Berlina + Diesel = 11500
```

Paso 1. Se construye **de adentro hacia afuera**: `Berlina` ← `Diesel` ← `Turbo`. La capa más externa (`Turbo`) es con la que habla el cliente.

Paso 2. `v.precio()` ejecuta `Turbo.precio()`: pide el precio al objeto envuelto (`Diesel`), suma 800.

Paso 3. `Diesel.precio()` pide el de `Berlina` (10000), suma 1500 → 11500. Luego Turbo suma 800 → **12300**.

Paso 4. `descripcion()` se arma igual: va concatenando de adentro hacia afuera.

Paso 5. Con herencia hubieran hecho falta 10 clases hoja (2 × 5). Con Decorator: **2 + 5 clases** y cualquier combinación se arma **en tiempo de ejecución**.

> **TRAMPA CLÁSICA** Para que sea Decorator y no una herencia más, el decorador debe **(1)** ser del mismo tipo que el componente y **(2)** tener una **referencia** a otro componente que recibe por constructor y a la que **delega**.

Ejercicio 6.4 · Observer: termómetro y pantallas

```
import java.util.Observable;  import java.util.Observer;
class Termometro extends Observable {
    private double temp;
    public void setTemp(double t) {
        this.temp = t;
        setChanged();               // marca "hubo un cambio"
        notifyObservers(t);         // llama a update() de cada observador
    }
}
class Pantalla implements Observer {
    private String nombre;
    public Pantalla(String nombre, Termometro t) { this.nombre = nombre; t.addObserver(this); }
    @Override public void update(Observable o, Object arg) {
        System.out.println(nombre + " muestra: " + arg + " grados");
    }
}
// main: t = new Termometro(); new Pantalla("Pantalla-1", t); p2 = new Pantalla("Pantalla-2", t);
//       t.setTemp(21.5);  t.deleteObserver(p2);  t.setTemp(30);
```

Salida real (verificada ejecutando el código):

```
Pantalla-2 muestra: 21.5 grados
```

```
Pantalla-1 muestra: 21.5 grados
Pantalla-1 muestra: 30.0 grados
```

Paso 1. Cada `Pantalla` se **registra** a sí misma en el constructor con `t.addObserver(this)` (es responsabilidad del observador vincularse con el observado).

Paso 2. `setTemp(21.5)` cambia el estado, marca `setChanged()` y llama a `notifyObservers(21.5)`: **ambas** pantallas reciben `update(...)`.

Paso 3. `deleteObserver(p2)` la da de baja: el segundo cambio (30) solo lo recibe `Pantalla-1`.

Paso 4. Detalle: la implementación de Java notifica en **orden inverso al de registro** (Pantalla-2 primero). **Nunca dependas del orden de notificación.**

> **TRAMPA CLÁSICA** Si te olvidás de `setChanged()`, `notifyObservers()` **no notifica a nadie** (la marca de "modificado" sigue en false).

Ejercicio 6.5 · State: el vehículo con contacto

El vehículo tiene tres estados: **Apagado**, **Parado** y **En marcha**. Las acciones son `contacto`, `acelerar` y `frenar`, y cada una actúa distinto según el estado (o no hace nada).

Estado actual	contacto()	acelerar()	frenar()
Apagado	→ Parado	ERROR	ERROR
Parado	→ Apagado	vel = 10 → En marcha	ya está parado
En marcha	ERROR (no se puede apagar)	vel + 10 (sigue En marcha)	vel = 0 → Parado

```
interface Estado { void contacto(); void acelerar(); void frenar(); }
class Vehiculo {                                    // CONTEXTO
    private Estado estado;  private int velocidad;
    public Vehiculo() { estado = new Apagado(this); }
    void setEstado(Estado e) { estado = e; }
    int getVelocidad() { return velocidad; }   void setVelocidad(int v) { velocidad = v; }
    public void contacto() { estado.contacto(); }  // delega en el estado actual
    public void acelerar() { estado.acelerar(); }
    public void frenar()   { estado.frenar(); }
}
class Apagado implements Estado {
    private Vehiculo v;  Apagado(Vehiculo v) { this.v = v; }
    public void contacto() { System.out.println("-> PARADO"); v.setEstado(new Parado(v)); }
    public void acelerar() { System.out.println("ERROR: esta apagado"); }
    public void frenar()   { System.out.println("ERROR: esta apagado"); }
}
class Parado implements Estado {
    private Vehiculo v;  Parado(Vehiculo v) { this.v = v; }
    public void contacto() { System.out.println("-> APAGADO"); v.setEstado(new Apagado(v)); }
    public void acelerar() { v.setVelocidad(10); System.out.println("-> EN MARCHA"); v.setEstado(new EnMarcha(v)); }
    public void frenar()   { System.out.println("ya esta parado"); }
}
class EnMarcha implements Estado {
    private Vehiculo v;  EnMarcha(Vehiculo v) { this.v = v; }
    public void contacto() { System.out.println("ERROR: en marcha, no se puede apagar"); }
    public void acelerar() { v.setVelocidad(v.getVelocidad() + 10); System.out.println("velocidad " + v.getVelocidad
    public void frenar()   { v.setVelocidad(0); System.out.println("-> PARADO"); v.setEstado(new Parado(v)); }
}
// main: v.acelerar(); v.contacto(); v.acelerar(); v.acelerar(); v.contacto(); v.frenar(); v.contacto();
```

Salida real (verificada ejecutando el código):

```
ERROR: esta apagado
-> PARADO
-> EN MARCHA
velocidad 20
ERROR: en marcha, no se puede apagar
-> PARADO
-> APAGADO
```

Paso 1. Arranca **Apagado**. `acelerar()` → ERROR.

Paso 2. `contacto()` → pasa a Parado. `acelerar()` → velocidad 10 y pasa a En marcha. Otro `acelerar()` → velocidad 20.

Paso 3. `contacto()` en marcha → ERROR. `frenar()` → vuelve a Parado. `contacto()` → Apagado.

Paso 4. Fijate qué no hay: ningún `if (estado == ...)` en `Vehiculo`. El "if" lo reemplaza el **polimorfismo**: cada estado sabe qué hacer.

Ejercicio 6.6 · Elegir el patrón correcto

Enunciado	Patrón	Justificación
"El sistema debe tener un único gestor de impresión accesible desde todos lados"	**Singleton**	Una sola instancia + acceso global
"Según el tipo de documento elegido por el usuario se crea un exportador distinto, pero el cliente no debe conocer las clases concretas"	**Factory**	Oculta la lógica de creación
"Un café puede llevar leche, crema, canela... en cualquier combinación y cada extra suma precio"	**Decorator**	Extras opcionales combinables; evita explosión de clases
"Un reporte siempre abre, procesa, formatea y cierra; solo el formato cambia entre PDF y Excel"	**Template Method**	Mismos pasos y orden; un paso varía por subclase
"Cuando cambia el stock, la pantalla, el mail de alerta y el log deben actualizarse solos"	**Observer**	Un sujeto, muchos dependientes notificados
"Un pedido se comporta distinto si está Pendiente, Enviado o Cancelado, y cambia de uno a otro"	**State**	Comportamiento según estado + transiciones

Mini-práctica: el café con extras. Verificado con código:

```
interface Bebida { String descripcion(); double precio(); }
class Cafe implements Bebida { public String descripcion() { return "Cafe"; }  public double precio() { return 2.0;
abstract class Extra implements Bebida { protected Bebida base; Extra(Bebida b) { base = b; } }
class Leche extends Extra { Leche(Bebida b){ super(b); }
    public String descripcion() { return base.descripcion() + " + leche"; }
    public double precio() { return base.precio() + 0.5; } }
class Crema extends Extra { Crema(Bebida b){ super(b); }
    public String descripcion() { return base.descripcion() + " + crema"; }
    public double precio() { return base.precio() + 0.8; } }
// main: new Crema(new Leche(new Leche(new Cafe())))
```

Salida real (verificada ejecutando el código):

```
Cafe + leche + leche + crema = 3.8
```

Cuenta: 2.0 (café) + 0.5 + 0.5 (dos leches) + 0.8 (crema) = **3.8**. Notá que un extra se puede **repetir**: eso es imposible de modelar con una jerarquía fija.

Autoevaluación: ¿qué tan firme estás?

Contestá primero **sin mirar** las respuestas (están al final de la guía). Si dudás en una, volvé al resumen.

- ☐ **1.** ¿Qué es un patrón de diseño y qué aporta "gratis"? ¿Qué NO aporta?
- ☐ **2.** ¿Qué tres elementos tiene un Singleton?
- ☐ **3.** ¿Qué problema resuelve Factory y qué principio usa?
- ☐ **4.** Explicá Template Method: ¿qué va concreto, qué abstracto y por qué las hijas no cambian el orden?
- ☐ **5.** ¿Qué es un hook?
- ☐ **6.** ¿Por qué el Decorator hereda del mismo tipo que lo decorado?
- ☐ **7.** Dos ventajas y dos desventajas de Decorator.
- ☐ **8.** En Observer de Java: ¿qué métodos se invocan para notificar y quién implementa `update`?
- ☐ **9.** ¿Cuál es el problema del `if` por estado y cómo lo resuelve State?

☐ **10.** ¿Qué es la inversión de control en un framework?

☐ **11.** Diferenciá un framework de una API/biblioteca.

Unidad 7 · Arquitectura en capas

Hasta acá vimos cómo diseñar **clases**; esta unidad muestra cómo organizar **todo el sistema** para que cada parte se pueda desarrollar, probar y cambiar sin romper las demás.

1. ¿Qué es la programación por capas?

Es una metodología en la que el objetivo primordial es la **separación (desacoplamiento) de las partes** de un sistema: cada parte, llamada **capa**, expone claramente las operaciones que puede realizar mediante un **API** (qué servicio ofrece y qué retorna, sin importar cómo está implementado). Pueden existir *n* capas, pero cada una debe tener una **responsabilidad única**. La separación más usada es de **tres capas**.

Ventajas: se programa más rápido y organizado, se cometen menos errores y son fáciles de ubicar y corregir, facilita el trabajo en equipo (cada uno trabaja en lo suyo sin pisarse) y las capas (o partes) se **reutilizan** (ej. la entidad CLIENTE sirve en cualquier aplicación que necesite clientes).

2. Las tres capas

Capa	Qué contiene	Se comunica con…	Ejemplos
Presentación (CP)	Todo lo que **ve el usuario**: formularios, menús, reportes. Atiende eventos y representa datos. Hace **validaciones simples** de formato que no requieren BD	Solo con **Negocio**	Fecha de venta no posterior a hoy; cantidad no negativa; número de factura presente
Lógica de negocio (CLN)	Las **reglas del negocio** (requerimientos funcionales), **entidades** y **procesos**; validaciones **complejas** o que requieren BD	Con **Presentación** (recibe pedidos, devuelve resultados) y con **Datos**	El cliente debe existir; no se vende a crédito a un moroso; no hay devoluciones pasadas las 72 horas
Acceso a datos (CAD)	Los comandos que hablan **directamente con la base de datos**: conectar, transacciones, procedimientos almacenados. No dice cómo se persiste	Solo con **Negocio**	Guardar usuario / obtener usuario

> **TIP DE PARCIAL** Una entidad (ej. EMPLEADO) en la capa de negocio necesita rutinas para **grabar**, **obtener por identificador**, **obtener todos** y **buscar el identificador por número de legajo**. Esas rutinas llaman a la capa de datos.

Independencia del origen de datos: como solo la CAD escribe comandos específicos del motor (ej. `EXECUTE PROCEDURE` en Firebird vs `EXEC` en SQL Server), se puede cambiar de motor tocando únicamente la CAD. Las demás capas reciben un **cursor** y ni se enteran.

> **OJO** Ni la CLN ni la CAD deben **mostrar mensajes** al usuario: si detectan un error devuelven un código o mensaje de error y es la **CP** quien lo muestra. Además, ni la CP ni la CLN deben conocer los **nombres reales de tablas, vistas o procedimientos**: eso es competencia exclusiva de la CAD.

3. Reglas de comunicación entre capas

Paso 1. **Responsabilidad única:** cada capa está perfectamente delimitada (presentación = eventos del cliente y representación; acceso a datos = guardar y obtener).

Paso 2. **Estructura jerárquica estricta:** cada capa **solo puede comunicarse con la que está inmediatamente debajo**, nunca al revés, y **no se pueden saltar capas**. Presentación → Negocio → Datos.

✓ Permitido	✘ No permitido
Presentación llama a Negocio	Datos llama a Presentación (sentido inverso)
Negocio llama a Datos	Presentación llama **directo** a Datos (salta la capa de negocio)
Cada capa expone su API	Una capa depende de detalles internos de otra

4. Capa vs nivel

Dos palabras que se confunden: **capa** es la división **lógica** de la solución (Presentación / Negocio / Datos); **nivel** es cómo esas capas se **distribuyen físicamente** en máquinas.

Despliegue	Se dice…
Presentación + Negocio + Datos en **1** computadora	Tres capas **y un nivel**
Presentación+Negocio en una máquina, Negocio+Datos en otra (2 equipos)	Tres capas **y dos niveles**
Cada capa en su propia computadora (3 equipos)	Tres capas **y tres niveles**

5. Cohesión y acoplamiento

	Cohesión	Acoplamiento
Qué mide	Que los módulos trabajen **coordinadamente** para alcanzar su objetivo	La **dependencia** entre módulos
Lo ideal	**Alta**	**Bajo** (idealmente módulos independientes)

6. Capas vs MVC

	Arquitectura en 3 capas	MVC
Naturaleza	Un **estilo arquitectónico**	Un **patrón de diseño**
Forma	**Lineal**: las capas no se saltan; todo pasa por la intermedia	**Triangular**: hay relación vista–controlador, controlador–modelo y vista–modelo
Se pueden combinar	Sí: **Controladores y Vistas** de MVC = nivel de **presentación**; **Modelo** = nivel de **negocio**; el acceso a datos queda igual	

Pasar de 3 capas a MVC: hay que cambiar la **capa de presentación**. Si la lógica está toda fuera de la presentación y las capas inferiores no dependen de ella, la conversión es relativamente simple: el HTML, estilos y JavaScript se reutilizan para las **Vistas**, las clases que viajan entre negocio y presentación pueden servir de **Modelo**, y los eventos pasan a ser **métodos de acción del Controlador**.

> **NOTA** Tema que el apunte deja **pendiente**: *cuándo* validar, lanzar excepciones o establecer un contrato. Con lo visto en las Unidades 3 y 5: la **CP valida formato**; la **CLN valida reglas de negocio** y lanza excepciones cuando aun cumpliéndose la precondición no puede cumplir la postcondición; el **contrato** define quién verifica qué.

Términos clave de la Unidad 7

Término	En una frase
Capa	División **lógica** de un sistema con responsabilidad única
Nivel	Distribución **física** de las capas en máquinas
CP / CLN / CAD	Presentación / Lógica de negocio / Acceso a datos
Jerarquía estricta	Cada capa solo habla con la inmediata inferior, sin saltear
Cohesión / acoplamiento	Cuánto colaboran los módulos / cuánto dependen unos de otros
MVC	Patrón Modelo–Vista–Controlador (relación triangular)

Ejercicio 7.1 · ¿En qué capa va cada cosa?

Elemento	Capa	Por qué
Validar que la cantidad vendida no sea negativa	Presentación	Validación simple de formato, sin acceder a la BD
Verificar que el cliente exista	Negocio	Requiere consultar datos (validación compleja)
`EXEC sp_guardarVenta` (procedimiento almacenado)	Acceso a datos	Comando específico del motor; solo la CAD lo conoce
Rechazar devoluciones pasadas las 72 h	Negocio	Regla del negocio
Mostrar el cartel "Cliente moroso"	Presentación	Solo la CP muestra mensajes; la CLN devuelve el error
Iniciar y finalizar una transacción	Acceso a datos	Interacción directa con la base
Un formulario de alta de cliente llama directamente a una consulta SQL	✘ Incorrecto	Salta la capa de negocio: viola la jerarquía estricta

Flujo completo: alta de usuario. (1) La **CP** captura los datos y valida formato; (2) llama a `altaDeUsuario(...)` en la **CLN**; (3) la CLN valida el nombre, aplica reglas y envía el mail; (4) pide a la **CAD** `guardarUsuario(...)`; (5) la CAD ejecuta el comando contra la BD; (6) el resultado **vuelve** CAD → CLN → CP y la CP lo muestra.

Autoevaluación: ¿qué tan firme estás?

Contestá primero **sin mirar** las respuestas (están al final de la guía). Si dudás en una, volvé al resumen.

- ☐ **1.** ¿Cuáles son las tres capas principales y qué responsabilidad única tiene cada una?
- ☐ **2.** Enunciá las dos reglas de comunicación entre capas.
- ☐ **3.** Diferenciá capa de nivel con un ejemplo.
- ☐ **4.** ¿Qué validaciones van en presentación y cuáles en negocio?
- ☐ **5.** ¿Por qué cambiar de motor de base de datos solo afecta a la capa de acceso a datos?
- ☐ **6.** ¿Qué capa puede mostrar mensajes al usuario?
- ☐ **7.** Diferenciá arquitectura de 3 capas y MVC.
- ☐ **8.** Definí cohesión y acoplamiento y decí qué se busca.

Atlas de comparaciones (repaso transversal)

Los cuadros que más se confunden, juntos y en versión compacta. Es la página para mirar el día anterior al examen.

A. ¿Herencia, interfaz o delegación?

	Herencia (`extends`)	Interfaz (`implements`)	Delegación
Relación	"es un"	"se comporta como" (contrato)	"tiene un" / "usa un"
Hereda implementación	Sí	No	No (reenvía)
Cantidad	Una sola clase	Varias	Cuantas quieras
Se ve el interior	Caja blanca	Solo el contrato	Caja negra
Úsala cuando…	El "es un" es obvio y hay código/estado común	Querés un comportamiento para clases no relacionadas	Reutilizás selectivamente o no se cumple Liskov

B. Sobrecarga, sobreescritura y ocultamiento: ¿cuándo se resuelven?

	Se decide…	Según…	Ejemplo
Sobrecarga	Al compilar	Cantidad y tipo de parámetros	`depositar(double)` / `depositar(double, String)`
Sobreescritura	Al ejecutar	La **clase real** del objeto	`animal.emitirSonido()`
Ocultamiento de atributo	Al compilar	El **tipo declarado** de la variable	`Padre p = new Hijo(); p.atributo1`

C. Contrato, aserción y excepción: ¿qué uso para qué?

Herramienta	Para qué	Cuándo se comprueba	Quién se hace cargo
Precondición	Lo que debe cumplirse antes de llamar	No se verifica dentro del método	Quien llama
Postcondición	Lo que queda cierto al terminar	Se promete (puede chequearse con assert)	El método
Invariante	Siempre cierto para la clase / el ciclo	Antes y después de cada método / iteración	Todos los métodos / el ciclo
Aserción	Control interno en desarrollo	Solo con `-ea`	El programador
Excepción	Aun cumplida la pre, no se puede cumplir la post	Siempre	Quien capture o propague

D. Clase abstracta vs interfaz

	Clase abstracta	Interfaz
Cuerpo de métodos	Concretos + abstractos	Solo abstractos (regla del apunte)
Estado	Atributos de instancia y constructores	Solo constantes (`public static final`)
Cantidad por clase	1	N
Vínculo	Jerarquía (herencia)	Sin jerarquía

E. Excepciones: las parejas que se confunden

Pareja	Diferencia
`throw` / `throws`	Lanza un objeto / declara en la cabecera lo que se propaga
`final` / `finally` / `finalize`	Modificador / bloque que siempre corre / método del recolector
Comprobada / no comprobada	Exige declarar o capturar / no lo exige (`RuntimeException`, `Error`)
`Error` / `Exception`	Problema grave de la JVM, no se trata / situación que la aplicación debe manejar
`catch` hija / padre	Primero la hija; si no, el hijo queda inalcanzable (error de compilación)

F. Copia: superficial vs profunda; igualdad vs identidad

Operación	Resultado
`y = x` (asignación)	Misma referencia: **un solo objeto**, dos nombres
`x.clone()` superficial	Objeto nuevo, pero los atributos objeto **se comparten**
`x.clone()` profundo	Objeto nuevo y sus atributos objeto también **clonados**
`x == y`	Compara **referencias** (¿es el mismo objeto?)

G. Patrones: cómo se distinguen entre sí

Patrón	Mecanismo principal	Cuándo cambia el comportamiento	Pregunta clave
Template Method	Herencia + método abstracto/hook	Al elegir la **subclase**	¿Los pasos y el orden son siempre los mismos?
Decorator	Composición (envolver)	En **ejecución**, al envolver	¿Hay extras opcionales combinables?
State	Composición + transiciones	En **ejecución**, al cambiar de estado	¿Hay `if` por estado en todos los métodos?
Observer	Lista de suscriptores + `update`	Cuando el sujeto notifica	¿Muchos deben enterarse de un cambio?
Factory	Método que crea (polimorfismo)	Al pedir el objeto	¿El cliente no debe conocer la clase concreta?
Singleton	Constructor privado + `getInstance`	—	¿Debe haber una sola instancia?

H. Capas: capa/nivel, capas/MVC

Par	Diferencia
Capa / nivel	División lógica / distribución física
3 capas / MVC	Estilo lineal / patrón triangular; MVC puede vivir en la capa de presentación y negocio
Cohesión / acoplamiento	Colaboración interna (alta) / dependencia externa (baja)

Hoja de última hora: frases para recordar

Tema	Regla en una línea
Paso de parámetros	Siempre por valor; con objetos se copia la **referencia**.
`static`	Pertenece a la clase; sin `this`; solo toca miembros static.
`final` + objeto	No cambia la referencia, **sí** el estado del objeto.
Agregación / composición	Rombo **blanco**: parte compartible / rombo **negro**: vida ligada.
Constructores en herencia	Primera línea `super(...)`; si el padre no tiene constructor sin parámetros, hay que llamarlo.
Reducir acceso	Prohibido al sobreescribir: solo se puede **ampliar**.
Late binding	**Métodos** por objeto real; **atributos** por tipo de variable.
Liskov	Donde va el padre puede ir el hijo sin cambiar la semántica.
Contratos	Pre: la garantiza el que llama (no se verifica). Post: la garantiza el método.
Al sobreescribir	Se puede **relajar la pre** y **fortalecer la post**.
Interfaz	Contrato sin implementación; una clase implementa varias; no se instancia.
ISP	Varias interfaces chicas > una grande.
Doble envío	En vez de `if instanceof`, el objeto le manda otro mensaje al parámetro.
`assert`	Desactivado por defecto; se activa con `-ea`; no para validar datos del usuario.
`catch`	Hijas antes que padres.
`finally`	Siempre corre si el `try` comenzó; un `return` en `finally` pisa al del `try`.
`throws` y override	No se pueden agregar excepciones comprobadas nuevas.
`clone()`	`super.clone()` primero; luego clonar a mano los atributos objeto.
Singleton	Constructor privado + atributo estático + `getInstance()`.
Template Method	Plantilla `final`; pasos variables abstractos; **hooks** opcionales.
Decorator	Mismo tipo que lo decorado + referencia por constructor + delega.
Observer	`setChanged()` y después `notifyObservers()`.
State	Una clase por estado; el contexto delega; sin `if` de estado.
Framework	Inversión de control: **él** te llama a vos.
Capas	Cada capa habla solo con la inmediata inferior; solo la CP muestra mensajes.

Simulacro de parcial (90 minutos)

Resolvelo **sin mirar nada**, con reloj. Después corregí con la clave de más abajo. Sugerencia de tiempo: 30 min teoría + 60 min práctica.

Parte teórica (respondé en 4 a 6 líneas cada una)

T1. Diferenciá **agregación** de **composición** con un ejemplo de cada una y sus símbolos UML.

T2. Explicá **enlace tardío** y qué ocurre con métodos y atributos en `Padre p = new Hijo();`.

T3. Definí **precondición** y **postcondición**. ¿Qué se puede hacer con ellas al sobreescribir un método?

T4. Compará **clase abstracta** e **interfaz** y explicá el **principio de segregación de interfaces**.

T5. Explicá el funcionamiento de `finally` y qué pasa si tiene un `return`. Diferenciá excepciones comprobadas de no comprobadas.

T6. ¿Qué problema resuelve **Decorator** frente a la herencia? Nombrá los participantes.

Parte práctica

P1. ¿Qué imprime?

```
class Animal {
    String nombre = "animal";
    String sonido() { return "..."; }
    String presentar() { return nombre + " dice " + sonido(); }
}
class Perro extends Animal {
    String nombre = "perro";
    @Override String sonido() { return "guau"; }
    String presentar() { return super.presentar() + " / " + nombre; }
}
// main:
Animal a = new Perro();
System.out.println(a.nombre);
System.out.println(a.sonido());
System.out.println(a.presentar());
Perro p = (Perro) a;
System.out.println(p.nombre);
```

P2. Encontrá el error de compilación en cada fragmento y corregilo.

```
// (a)
class Padre { public void hablar() { } }
class Hijo extends Padre { private void hablar() { } }
// (b)
try { leer(); }                       // leer() declara IOException
catch (Exception e)   { }
catch (IOException e) { }
// (c)
class Vehiculo { Vehiculo(String patente) { } }
class Auto extends Vehiculo { Auto(int puertas) { } }
```

P3. Diseño. Una tienda online maneja pedidos. Un pedido pasa por los estados **Pendiente → Enviado → Entregado** y también puede **Cancelarse**; el método `cancelar()` hace cosas distintas según el estado (en Entregado no se puede). Cada vez que cambia el estado deben enterarse automáticamente el **cliente** (por mail) y el **depósito**. ¿Qué patrones usarías y cómo armarías las clases? Hacé un diagrama de clases.

P4. Capas. Ubicá en capa Presentación, Negocio o Datos: (a) comprobar que el stock no sea negativo al tipear; (b) rechazar la venta si el cliente tiene deuda; (c) abrir una transacción; (d) mostrar "Venta rechazada"; (e) guardar la venta.

Clave de respuestas

Simulacro: respuestas modelo

Ítem	Respuesta modelo
T1	**Agregación**: rombo blanco; el todo contiene partes que pueden compartirse y sobreviven al todo (Persona–Categoría). **Composición**: rombo negro; la parte no se comparte y vive y muere con el todo (Persona–Domicilio).
T2	Es resolver en **ejecución** qué método se invoca según la **clase real** del objeto. Con `Padre p = new Hijo()`: los **métodos sobreescritos** ejecutan la versión del Hijo; los **atributos** se leen según el **tipo declarado** (Padre); lo propio del Hijo no es accesible sin cast.
T3	Pre: condiciones previas que garantiza **quien llama**; Post: resultado o estado posterior que garantiza **el método**. Al sobreescribir se pueden **relajar las pre y fortalecer las post** (Liskov).
T4	Abstracta: métodos concretos y abstractos, atributos, constructores, una por clase, forma jerarquía. Interfaz: solo contrato (y constantes), varias por clase, sin jerarquía. ISP: mejor varias interfaces específicas; nadie debe depender de métodos que no usa.
T5	`finally` corre siempre que el `try` comenzó; un `return` en `finally` anula el del `try`/`catch`. Comprobadas (`Exception` menos `RuntimeException`): hay que capturar o declarar. No comprobadas (`RuntimeException`, `Error`): no es obligatorio.
T6	Agrega responsabilidades dinámicamente evitando la **explosión de clases**. Participantes: Component, ComponenteConcreto, Decorator (hereda de Component y tiene una referencia a otro Component) y DecoratorConcreto.

P1 (salida verificada):

Salida real (verificada ejecutando el código):

```
animal
guau
animal dice guau / perro
perro
```

Paso 1. `a.nombre`: atributo → **tipo declarado** (Animal) → `animal`.

Paso 2. `a.sonido()`: método sobreescrito → **objeto real** (Perro) → `guau`.

Paso 3. `a.presentar()` ejecuta `Perro.presentar()`, que llama a `super.presentar()` (la de Animal). Dentro de `Animal`, `nombre` es el de Animal (`animal`) y `sonido()` sigue siendo polimórfico (`guau`) → `animal dice guau`. Luego Perro agrega `/` y **su** `nombre` (`perro`).

Paso 4. Con el cast, `p.nombre` pasa a mirar al Perro → `perro`.

P2: (a) Se reduce el acceso (público→privado): hacer `public` el método del hijo. (b) `Exception` es padre de `IOException`: invertir el orden (primero `IOException`). (c) Falta `super(patente)`: agregar el parámetro y llamarlo, o crear un constructor sin argumentos en `Vehiculo`.

P3: State para el pedido: interfaz `EstadoPedido` (`cancelar()`, `avanzar()`), clases `Pendiente`, `Enviado`, `Entregado`, `Cancelado`, y contexto `Pedido` que guarda el estado actual y delega. **Observer** para las notificaciones: `Pedido` es Observable (llama a `setChanged()` y `notifyObservers(...)` al cambiar de estado); `Cliente`/`ClienteMail` y `Deposito` son Observers que implementan `update`. En el diagrama: `Pedido` ◆→ `EstadoPedido` (interfaz, con 4 implementaciones), `Pedido` hereda de `Observable`, y `ClienteMail`/`Deposito` implementan `Observer`.

P4: (a) Presentación (validación simple de formato). (b) Negocio (regla de negocio). (c) Datos (transacción). (d) Presentación (solo ella muestra mensajes). (e) Datos (persistencia), invocada desde Negocio.

Respuestas de las autoevaluaciones

Unidad 1 · Fundamentos de POO

#	Respuesta
1	La clase es la definición/molde (Auto); el objeto es una instancia concreta con su propio estado (miAuto, rojo, 2018).
2	Reservar memoria, inicializar atributos por defecto, ejecutar el constructor, asignar la referencia a la variable.
3	Falso. El pasaje es siempre por valor; con objetos el valor copiado es la referencia, por eso se puede modificar el objeto pero no reasignar la variable del llamador.
4	Porque cualquiera podría dejar al objeto en un estado inválido (edad = -7). La validación es responsabilidad de la propia clase, en los setters/constructor.
5	No. La signatura es nombre + parámetros; el retorno no forma parte, así que el compilador no podría distinguirlos.
6	Puede acceder solo a miembros static de la clase; no tiene `this`. Se invoca sobre la clase, sin instancia.
7	El estado sí (se puede modificar); la referencia no (no se puede reasignar).
8	Agregación: rombo blanco, las partes pueden compartirse y sobreviven al todo. Composición: rombo negro, las partes no se comparten y mueren con el todo.
9	Se invoca antes de que el GC libere el objeto; sirve para liberar recursos no Java (archivos, etc.). El GC ya maneja la memoria automáticamente.
10	Cuando A y B son instancias de la misma clase: el control de acceso es por clase, no por objeto.

Unidad 2 · Herencia y abstractas

#	Respuesta
1	El contrato/tipo (puede usarse polimórficamente donde se usa la superclase) y la implementación (campos y métodos accesibles).
2	Sobrecarga: mismo nombre, distinta signatura. Sobreescritura: misma signatura en la subclase, nueva implementación. Ocultamiento: atributo de la subclase con el mismo nombre que el del padre.
3	Para delegar en el padre la inicialización del estado heredado; debe ser la primera sentencia.
4	Público→protegido no (reduce acceso). Protegido→público sí (amplía acceso).
5	Clase: no se extiende. Método: no se redefine. Variable: no cambia su valor luego de inicializarse.
6	Cuando la relación es "tiene un" y no se cumple Liskov, o cuando querés reutilizar selectivamente sin necesidad de tratar al objeto como el padre.
7	No se puede instanciar. Sí puede tener métodos concretos y atributos.
8	Debe declararse también abstracta; si es concreta, no compila.
9	Diseñar clases por separado, descubrir lo común, abstraerlo en una superclase, armar la jerarquía, quitar lo común de las subclases y verificar el "es un".
10	java.lang.Object; toString().

Unidad 3 · Polimorfismo y contratos

#	Respuesta
1	Cualquier objeto de una subclase debe poder usarse donde se espera uno de la superclase sin alterar la corrección del programa.
2	De sobrecarga, paramétrico y de inclusión (redefinición/subtipado).
3	El método del Hijo (clase real, enlace tardío); el atributo de Padre (tipo declarado de la variable).
4	Es la conexión de una llamada con su método en tiempo de ejecución, según el objeto real.
5	Subclases, clases del mismo paquete y referencias del mismo tipo (o subtipo) de la clase que accede.
6	Pre: condiciones previas, las garantiza quien llama. Post: resultado/estado posterior, lo garantiza el método.
7	No debe verificarla. La excepción aparece cuando, cumpliéndose la pre, es imposible cumplir la post.
8	Precondiciones: solo relajarlas. Postcondiciones: se pueden fortalecer.
9	La responsabilidad va a la clase que posee la información: Club agrega socios (dueño de la lista), Socio agrega autorizados.
10	Porque el tipo declarado de `p` es Padre, que no conoce `atributo3`; hace falta un cast a Hijo.

Unidad 4 · Interfaces y aserciones

#	Respuesta
1	Ambigüedad al heredar un método de A por dos caminos (B y C) hacia D. Java no permite herencia múltiple de clases; usa interfaces (herencia de contrato sin implementación).
2	Abstracta: tiene métodos concretos, atributos de instancia y constructores, se extiende una sola. Interfaz: solo contrato (y constantes), no tiene constructores ni estado de instancia, se pueden implementar varias.
3	No se puede instanciar; sí se puede declarar una variable y asignarle un objeto de una clase que la implemente.
4	Porque todo campo de una interfaz es implícitamente public static final; el campo es estático igualmente.
5	Los clientes no deben depender de interfaces que no usan; mejor varias interfaces específicas. Ej.: separar Worker en Trabajador y Descansador para que un Robot no implemente descansar().
6	Resolver comportamiento que depende de receptor y argumento sin if/switch por clase. Ventaja: extensión por agregado, código más mantenible. Desventaja: una nueva clase de parámetro exige un método secundario en las otras.
7	De clase: propiedades que toda instancia cumple siempre. De bucle: condición verdadera antes y después de cada iteración.
8	Con la opción -ea de la JVM; se lanza AssertionError (un Error que no debería capturarse).
9	No pueden usar el valor antiguo de un atributo, no expresan cuantificadores/condiciones globales, se comprueban solo en ejecución, no se heredan y están desactivados por defecto.
10	compareTo: negativo si el receptor es menor, cero si es igual, positivo si es mayor.

Unidad 5 · Excepciones y clone

#	Respuesta
1	Error: problemas graves de la JVM, no se tratan. Exception: situaciones que la app debe manejar (comprobadas). RuntimeException: errores del programador, no comprobadas.
2	RuntimeException y Error (y sus subclases).
3	Capturarla y gestionarla; capturarla y lanzar otra propia declarada en su throws; declararla en su throws y propagarla.
4	Porque se atrapa en el primero que coincide; si el padre va primero, el hijo es inalcanzable y no compila.
5	Siempre que el try haya comenzado (con o sin excepción, con o sin return). Si tiene return, prevalece sobre el del try/catch.
6	No. Puede lanzar las mismas o subtipos.
7	throw lanza un objeto excepción (sentencia). throws declara en la cabecera las comprobadas que el método propaga.
8	final: modificador (constante, no redefinible, no extensible). finally: bloque que siempre corre tras try. finalize: método invocado por el GC antes de liberar un objeto.
9	Copia valores de los atributos; si son objetos, copia la referencia, así original y clon comparten objetos internos y se afectan mutuamente.
10	Para obtener un objeto del tipo correcto también cuando lo invoca una subclase; con new se crearía siempre la clase base.
11	Admitir, admitir condicionalmente, prohibir (lanzando siempre CloneNotSupportedException).

Unidad 6 · Patrones y frameworks

#	Respuesta
1	Una solución probada a un problema recurrente. Aporta experiencia, vocabulario común y documentación más clara; no aporta creatividad ni garantiza un buen diseño por sí solo.
2	Constructor privado, atributo estático privado de su propio tipo y método estático de acceso (getInstance) que crea la instancia si es null.
3	Crear objetos sin exponer la lógica de creación al cliente, devolviendo una referencia por una interfaz común; usa polimorfismo.
4	Los pasos comunes concretos, los variables abstractos; el método plantilla (a menudo final) define el orden, por eso las hijas solo implementan los pasos.
5	Un método de la clase base con comportamiento por defecto que las hijas pueden sobrescribir opcionalmente para agregar un paso.
6	Para que sean intercambiables (Liskov), no para reutilizar código.
7	Ventajas: más flexible que herencia, evita la explosión de clases. Desventajas: muchos objetos pequeños, difícil de depurar, combinaciones inválidas posibles.
8	setChanged() y luego notifyObservers(); update lo implementa el Observer.
9	Cada método se llena de condicionales; State encapsula cada estado en una clase y el contexto delega en el estado actual, que hace la transición.
10	El flujo de control lo dicta el framework, que llama al código del usuario (a diferencia de una biblioteca, que es llamada por el usuario).
11	Framework: esqueleto extensible con arquitectura e inversión de control; biblioteca/API: funciones que el programa invoca.

Unidad 7 · Arquitectura en capas

#	Respuesta
1	Presentación (interfaz con el usuario), Lógica de negocio (reglas, entidades, procesos) y Acceso a datos (comunicación con la BD).
2	Responsabilidad única por capa y estructura jerárquica estricta (solo con la inmediata inferior, sin saltear ni ir en sentido inverso).
3	Capa: división lógica; nivel: distribución física. Tres capas en dos computadoras = tres capas y dos niveles.
4	Presentación: simples de formato sin BD. Negocio: complejas o que requieren BD y las reglas del negocio.
5	Porque solo la CAD escribe comandos específicos del motor; las demás reciben cursores y no conocen tablas ni procedimientos.
6	Solo la de presentación; las otras devuelven códigos o mensajes de error.
7	3 capas: estilo arquitectónico lineal. MVC: patrón de diseño con relación triangular entre vista, controlador y modelo. Se pueden combinar.
8	Cohesión: colaboración coordinada de los módulos (alta). Acoplamiento: dependencia entre módulos (bajo).

Glosario de términos clave (A–Z)

Término	Definición
Abstracción	Suprimir detalles para quedarse con lo esencial.
Acoplamiento	Grado de dependencia entre módulos (se busca bajo).
Agregación	Parte-todo débil; las partes pueden compartirse y sobreviven (rombo blanco).
Aserción	Condición `assert` comprobada solo con `-ea`.
Capa / nivel	División lógica / distribución física del sistema.
Clase abstracta	No instanciable; define comportamiento común y métodos abstractos.
Clonar	Duplicar un objeto con `clone()`; puede ser superficial o profunda.
Cohesión	Qué tan coordinados trabajan los elementos de un módulo (se busca alta).
Composición	Parte-todo fuerte; vida ligada (rombo negro).
Constructor	Inicializa un objeto; mismo nombre que la clase, sin retorno.
Contrato	Pre y postcondiciones que definen compromisos de un método.
Decorator	Patrón que añade responsabilidades envolviendo un objeto.
Delegación	Una clase contiene a otra y le reenvía parte del trabajo.
Dependencia	Una clase usa a otra sin tenerla como atributo.
Doble envío	Resolver según receptor y argumento con un segundo mensaje.
Encapsulamiento	Ocultar la implementación; mostrar el contrato.
Enlace tardío	Elegir el método en tiempo de ejecución según el objeto real.
Excepción	Objeto que representa un evento que interrumpe el flujo normal.
Factory	Patrón creacional que oculta la lógica de creación.
`final`	Constante, método no redefinible o clase no extensible.
`finally`	Bloque que siempre se ejecuta tras un `try`.
Framework	Esqueleto extensible con inversión de control.
Garbage collector	Libera objetos sin referencias.
Herencia	Relación "es un": la subclase hereda contrato e implementación.
Hook	Paso opcional con comportamiento por defecto en Template Method.
Interfaz	Contrato abstracto implementable por varias clases.
Invariante	Propiedad siempre cierta (de clase) o por iteración (de bucle).
Liskov	Donde va un padre puede ir un hijo sin alterar la semántica.
Observer	Patrón de notificación a dependientes cuando cambia un sujeto.
Ocultamiento	Atributo del hijo que oculta al del padre.
Polimorfismo	Un mismo mensaje con distinto comportamiento según el objeto.
Postcondición	Lo que el método garantiza al terminar.
Precondición	Lo que quien llama debe garantizar antes.
Protocolo	Conjunto de mensajes que entiende un objeto.
`protected`	Acceso para subclases, paquete y referencias del mismo tipo.
Signatura	Nombre + parámetros del método.
Singleton	Una única instancia con acceso global.
SOLID	Cinco principios de diseño OO (S, O, L, I, D).

Término	Definición
State	Patrón: una clase por estado; el contexto delega.
`static`	Miembro de la clase, no de la instancia.
`super`	Acceso a lo heredado: método, atributo o constructor del padre.
Template Method	Algoritmo fijo con pasos variables definidos por subclases.
UML	Lenguaje estándar para modelar sistemas OO.